

MLOps and Deployment

An interview-focused guide to taking machine-learning systems from an idea to reliable production operation. Every topic is self-contained and follows the same study structure. Examples use small, inspectable building blocks; production systems should add authentication, secret management, access control, and organization-specific infrastructure.

ML Project Lifecycle

1. Overview

The ML project lifecycle is the repeatable path from a business problem to a monitored, improved, or retired model. Unlike ordinary software, an ML system depends on code, data, learned parameters, and its operating environment. A model can therefore fail even when the application code has not changed.

Typical phases are problem framing, data work, experimentation, validation, deployment, monitoring, feedback, retraining, and retirement. The lifecycle is used in fraud detection, forecasting, search, recommendations, computer vision, NLP, and generative-AI products.

2. Intuition

Treat an ML product like a living supply chain. Raw data enters, transformations manufacture features, training produces an artifact, deployment delivers it, and monitoring checks whether real users receive the promised value. A high offline score is only an intermediate quality check, not the final product.

3. Prerequisites

- Supervised/unsupervised learning, train-validation-test splits, and metrics.
- Python, Git, APIs, containers, databases, and basic cloud concepts.
- Data leakage, distribution shift, privacy, and software testing.
- Business KPIs, latency/availability requirements, and cost constraints.

4. Core Concepts

Subtopic	Meaning and example	Why it matters	Interview angle
Problem framing	Convert "reduce churn" into "rank accounts likely to churn in 30 days"	Defines target, horizon, action, and success	Distinguish model metric from business KPI
Data contract	Schema, units, allowed values, freshness, ownership	Prevents silent upstream breakage	How would you validate training/serving data?
Baseline	Rule, constant predictor, or current system	Shows whether ML adds value	Why start with a simple model?

Subtopic	Meaning and example	Why it matters	Interview angle
Experimentation	Controlled changes to data, features, model, and parameters	Makes results comparable	What must an experiment record?
Validation	Offline, slice, robustness, bias, and load evaluation	Averages can hide unsafe failures	Which gates block deployment?
Deployment	Package and expose a version through batch or online inference	Connects artifact to users	Compare rollout strategies
Monitoring loop	Observe inputs, outputs, service, and outcomes	Detects decay and incidents	What can be measured without labels?
Retirement	Remove an obsolete model and preserve audit history	Reduces cost and risk	When should a model be retired?

5. Algorithm / Working Process

1. Define the decision, prediction unit, horizon, constraints, owner, and measurable KPI.
2. Write a labeling rule and data contract; inspect coverage, bias, leakage, and legal constraints.
3. Build a non-ML and simple-ML baseline.
4. Split data as production will unfold, often by time, user, or geography.
5. Create a reproducible pipeline; track code, data, parameters, environment, and metrics.
6. Evaluate overall, by important slices, under stress, and against operational budgets.
7. Register an approved immutable artifact with lineage and signatures.
8. Deploy with canary, shadow, or A/B controls and a rollback path.
9. Monitor service health, data quality, model behavior, business outcomes, and cost.
10. Investigate alerts, retrain only when justified, and eventually retire the system.

Inputs include the business objective, historical data, constraints, and feedback. Outputs include a decision service or prediction dataset plus documentation, lineage, dashboards, and runbooks.

6. Mathematical Foundation

Offline empirical risk is

$$\hat{R}(f) = (1/n) \sum_i L(f(x_i), y_i)$$

but production chooses a model under multiple constraints:

```

maximize    expected_business_value(f)
subject to  latency_p99 <= L, cost/request <= C,
            recall(group g) >= r_g, availability >= A

```

Expected deployment value can be estimated as

$$\begin{aligned} EV = & P(\text{action succeeds} \mid \text{model}) * \text{benefit} \\ & - P(\text{harm} \mid \text{model}) * \text{harm_cost} \\ & - \text{serving_cost} - \text{maintenance_cost} \end{aligned}$$

The test set estimates generalization only for the sampled distribution. If production follows a different distribution, the estimate is biased.

7. Practical Implementation

```
from dataclasses import dataclass, asdict
from pathlib import Path
import hashlib, json, platform

@dataclass(frozen=True)
class RunManifest:
    code_commit: str
    data_uri: str
    data_sha256: str
    parameters: dict
    metrics: dict
    python: str = platform.python_version()

def sha256(path: str) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for block in iter(lambda: f.read(1 << 20), b''):
            h.update(block)
    return h.hexdigest()

def save_manifest(data_path: str, commit: str, params: dict, metrics: dict):
    manifest = RunManifest(commit, data_path, sha256(data_path), params, metrics)
    Path("artifacts").mkdir(exist_ok=True)
    Path("artifacts/run.json").write_text(
        json.dumps(asdict(manifest), indent=2), encoding="utf-8"
    )
```

8. Code Explanation

RunManifest records the minimum lineage needed to identify a result. The checksum detects changed bytes even if a filename is reused. Freezing the dataclass prevents accidental mutation, and JSON keeps the manifest human-readable. A production tracker would additionally store image digest, feature definitions, model signature, owner, timestamps, and artifact URIs.

9. Training / Evaluation

Use a split that matches deployment: temporal splits for forecasting, group splits for repeated users, and geographic holdouts for expansion. Fit preprocessing only on training data. Compare against a baseline; report uncertainty and slice metrics. Tune on validation data, touch the test set once, then run shadow/load tests before rollout. Diagnose underfitting through poor train and validation performance; diagnose overfitting through a widening train-validation gap.

10. Complexity and Cost

Lifecycle cost is dominated by data labeling/quality, repeated training, serving, monitoring, and human incident response. Estimate total cost of ownership rather than GPU training alone. Pipeline time is approximately the sum of extraction, feature computation, training, evaluation, packaging, and deployment; parallelize only independent stages. Store large artifacts in object storage, not Git.

11. Common Use Cases

- Fraud models with delayed chargeback labels.
- Recommendation rankers with online experiments.
- Demand forecasts retrained on a schedule.
- Vision inspection with edge deployment.
- LLM/RAG systems evaluated for quality, safety, latency, and cost.

12. Common Mistakes

- Starting with a model before defining the action and KPI.
- Random splitting time-dependent or user-dependent data.
- Using the test set during tuning.
- Tracking model files without data/code lineage.
- Ignoring slice quality, calibration, latency, or business impact.
- Retraining automatically without validation and promotion gates.

13. Edge Cases / Limitations

Labels may arrive months late, interventions can change labels, and feedback loops can make observed data endogenous. Rare safety events are poorly estimated by average metrics. Privacy rules may prohibit storing raw examples. Some domains require human approval, explainability, or deterministic fallback rather than full automation.

14. Variations

- CRISP-DM emphasizes business/data understanding; useful for placements.
- Continuous training automatically rebuilds candidates; useful only with reliable labels and gates.
- Human-in-the-loop routes uncertain or high-risk cases to reviewers.
- LLMOps adds prompt, retrieval corpus, evaluator, guardrail, and token-cost versioning.

15. Related Topics

Experiment tracking captures trials; versioning identifies code/data/models; a registry governs promotion; CI/CD automates checks and delivery; monitoring closes the feedback loop. DataOps focuses upstream, DevOps focuses software delivery, and MLOps joins both with model-specific validation.

16. Interview Questions

1. **Why is ML delivery harder than ordinary software?** Behavior depends on data and learned parameters as well as code.
2. **What should be framed first?** Decision, prediction target/unit, horizon, action, KPI, constraints, and owner.
3. **Why build a baseline?** It quantifies incremental value and exposes unnecessary complexity.
4. **Offline metric versus KPI?** AUC measures ranking; retained revenue measures business outcome.
5. **How do you avoid leakage?** Use point-in-time features and fit all transformations only on training data.
6. **What are deployment gates?** Required quality, fairness, security, latency, cost, and robustness thresholds.
7. **When retrain?** When verified drift or performance decay justifies cost—not merely on every alert.
8. **What enables rollback?** Immutable artifacts, backward-compatible interfaces, traffic control, and retained prior versions.
9. **What if labels are delayed?** Monitor proxies immediately and compute performance when labels mature.
10. **When retire a model?** When value no longer covers risk/cost, the task disappears, or a replacement is stable.

17. Practice Tasks

- Coding: emit a lineage manifest with checksums.
- Dataset project: build a temporal churn pipeline with a baseline and slice report.
- Experiment: compare random and temporal validation estimates.
- Debugging: find preprocessing leakage in a notebook.
- Extension: add an approval gate and rollback runbook.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Churn lifecycle	Train, register, serve, monitor churn risk	Python, MLflow, FastAPI; Telco Churn	End-to-end ownership
Vision quality gate	Detect defects and shadow-test at an edge line	PyTorch, Docker; MVTec AD	CV plus operations
RAG release pipeline	Version corpus/prompt and evaluate every release	DVC, MLflow; public QA corpus	Modern LLMOps evidence

19. Quick Revision

- Key idea: ML is a monitored decision lifecycle, not a one-time model.
- Main formula: maximize expected value under quality, latency, cost, and risk constraints.
- Metrics: model, slice, service, business, and cost metrics.
- Common traps: leakage, weak baselines, stale data, unowned alerts.
- Interview one-liner: “I make code, data, configuration, artifact, and deployment lineage reproducible, then close the loop with guarded rollout and monitoring.”

20. Final Cheat Sheet

Item	Summary
Definition	Problem-to-retirement process for an ML system
Input/output	Objective + data → governed prediction product
Main steps	Frame, data, baseline, train, validate, register, deploy, monitor, improve
Hyperparameters	Model parameters plus thresholds, windows, and rollout percentages
Metrics	Offline quality, slices, latency, errors, drift, KPI, cost
Pros/cons	Repeatability and safety / operational overhead
Best use	Any model that affects real users or recurring decisions

Experiment Tracking

1. Overview

Experiment tracking records each training/evaluation run: parameters, metrics, artifacts, code revision, dataset identity, environment, and notes. It answers “what changed, what won, and can we reproduce it?” Teams use it for hyperparameter search, ablations, audit trails, and handoff.

2. Intuition

It is a laboratory notebook with automatic measurements. Naming a file `final_model_v7.pkl` records almost nothing; a run record connects that artifact to the exact recipe and evidence that produced it.

3. Prerequisites

Metrics, hyperparameters, validation design, file/object storage, Git commits, serialization, and basic database concepts.

4. Core Concepts

Subtopic	Meaning / example	Why it matters	Interview angle
Run	One execution with a unique ID	Atomic comparison unit	Run versus experiment?
Experiment	Related set of runs	Organizes a hypothesis	How do you name/group runs?
Parameters	Inputs such as <code>depth=6</code>	Explain configuration	Parameter versus metric?
Metrics	Outputs such as <code>val_f1=0.82</code> , often by step	Compare results and curves	Best metric or last metric?

Subtopic	Meaning / example	Why it matters	Interview angle
Artifacts	Model, plots, predictions, confusion matrix	Preserve evidence	Where store large artifacts?
Tags	Owner, branch, task, dataset version	Search and governance	Required tags?
Lineage	Links code, data, features, environment, model	Reproduction and audit	What is insufficient about Git SHA alone?

5. Algorithm / Working Process

Define a hypothesis and primary metric; capture immutable context before training; log parameters; stream step metrics; write artifacts; mark status; compare only runs with compatible data/splits; promote a selected artifact through a registry. Failed runs should remain visible because they contain diagnostic evidence.

6. Mathematical Foundation

For run r with configuration θ_r and validation score m_r , naive selection is

$$r^* = \operatorname{argmax}_r m_r$$

Repeatedly selecting on the same validation set introduces selection bias. Report test performance only after selection. For noisy runs, log mean and uncertainty:

$$\begin{aligned} \text{mean} &= (1/k) \sum_j m_j \\ \text{SE} &= s / \sqrt{k} \\ \text{95\% CI} &\text{ approximately } \text{mean} \pm 1.96 * \text{SE} \end{aligned}$$

7. Practical Implementation

```
import json, time, uuid
from pathlib import Path

class LocalTracker:
    def __init__(self, root="runs"):
        self.path = Path(root) / uuid.uuid4().hex
        self.path.mkdir(parents=True)
        self.record = {"started_at": time.time(), "status": "RUNNING"}

    def log(self, **fields):
        self.record.update(fields)
        (self.path / "run.json").write_text(
            json.dumps(self.record, indent=2), encoding="utf-8"
        )
```

```
tracker = LocalTracker()
try:
    tracker.log(params={"max_depth": 5}, data_version="sha256:abc")
    # model.fit(X_train, y_train)
    tracker.log(metrics={"validation_f1": 0.84}, status="FINISHED")
except Exception as exc:
    tracker.log(status="FAILED", error=repr(exc))
    raise
```

8. Code Explanation

Each run receives an immutable directory. Parameters, data version, metrics, and lifecycle status share one record. The exception path persists failures and re-raises them. Real trackers add concurrent writes, authentication, remote artifact stores, step histories, UI comparison, and retention policies.

9. Training / Evaluation

Log split strategy and seed, not only scores. Save learning curves, calibration, per-slice metrics, predictions or their hashes, runtime, memory, and cost. Compare runs on the same dataset/split/evaluator version. For stochastic training, repeat important candidates across seeds.

10. Complexity and Cost

Scalar logs are cheap; per-example predictions, checkpoints, and media dominate storage and network traffic. Logging every batch can slow training, so buffer or sample. Retention policies should preserve promoted/audited runs while expiring redundant checkpoints.

11. Common Use Cases

Hyperparameter tuning, feature ablations, deep-learning curves, prompt/RAG evaluations, cross-team model selection, regulatory evidence, and failed-run diagnosis.

12. Common Mistakes

Manual run names, mutable artifacts, missing dataset/split version, incomparable metrics, logging secrets or personal data, choosing on the test set, and deleting failed runs.

13. Edge Cases / Limitations

Offline workers may need queued logs; distributed jobs can duplicate steps; mutable external tables break lineage; proprietary data may not be copied into artifacts. A tracker records evidence but does not make an invalid experiment valid.

14. Variations

Local CSV/JSON is enough for solo work; MLflow/W&B provide managed comparison; TensorBoard specializes in curves/media; an SQL metadata store supports custom governance. Placements should know the conceptual schema, not only a vendor UI.

15. Related Topics

Tracking stores run observations; version control identifies inputs; a registry manages deployable model versions. Hyperparameter optimization creates tracked runs, while reproducibility tests whether their lineage is sufficient.

16. Interview Questions

1. **What should every run log?** Code, data, split, environment, parameters, metrics, artifacts, status, time, and owner.
2. **Parameter versus metric?** A parameter is chosen before execution; a metric is observed.
3. **Artifact versus model registry entry?** Artifact is a file; registry entry adds identity, stage/alias, metadata, and governance.
4. **Why log failed runs?** They reveal bad configurations, infrastructure failures, and wasted search regions.
5. **How compare stochastic models?** Multiple seeds and confidence intervals.
6. **Why is dataset path insufficient?** Bytes at the path may change.
7. **How track distributed training?** One logical run; rank zero logs summaries, workers log carefully namespaced diagnostics.
8. **How avoid tracker bottlenecks?** Buffer, sample, batch uploads, and store large artifacts remotely.
9. **What causes misleading comparisons?** Different splits, preprocessing, evaluator versions, or label windows.
10. **What should never be logged?** Secrets and unrestricted sensitive records.

17. Practice Tasks

- Implement a JSON tracker with run IDs and failure status.
- Track five classifiers on one fixed split.
- Repeat the winner over five seeds.
- Diagnose why two "identical" runs differ.
- Add artifact hashes and a searchable summary table.

18. Project Ideas

Project	Description	Stack/dataset	Resume value
Experiment dashboard	Compare parameters, scores, runtime	MLflow, sklearn; Adult	Reproducible experimentation
CV ablation lab	Track augmentation/backbone studies	PyTorch; CIFAR-10	Research discipline
RAG evaluator	Track retriever/prompt/corpus variants	MLflow; BEIR subset	LLM evaluation

19. Quick Revision

Key idea: every score needs a reproducible recipe. Main selection equation: argmax validation score, followed by untouched test evaluation. Metrics include quality, uncertainty, runtime, memory, and cost. Trap:

comparing runs built from different data. Interview one-liner: “A run links immutable inputs to measured outputs and artifacts.”

20. Final Cheat Sheet

Item	Summary
Definition	Structured record of ML trials
Input/output	Configuration + lineage → metrics + artifacts
Steps	Start, capture context, log, finish/fail, compare
Key controls	Run ID, dataset/split/evaluator versions, seed
Pros/cons	Traceability / storage and process overhead
Best use	Tuning, ablations, collaboration, audits

Model Versioning

1. Overview

Model versioning assigns immutable identities to trained artifacts and their metadata. It lets teams reproduce, compare, deploy, roll back, audit, and retire exact model builds. A useful version includes weights plus preprocessing, signature, dependencies, and lineage.

2. Intuition

A model is a compiled product, not merely “the latest pickle.” Like a pharmaceutical batch, each release needs a batch number, recipe, tests, and destination history.

3. Prerequisites

Serialization, semantic versioning, Git, checksums, object storage, preprocessing pipelines, model signatures, and deployment basics.

4. Core Concepts

Concept	Meaning	Why it matters	Interview angle
Immutable artifact	Bytes never overwritten	Exact rollback/audit	Why not replace latest.pkl?
Content hash	Digest such as SHA-256	Detects byte-level identity	Version number versus hash
Signature	Input/output names, types, shapes	Prevents serving mismatch	How validate compatibility?

Concept	Meaning	Why it matters	Interview angle
Lineage	Training run, code, data, config	Explains provenance	Can weights reproduce training?
Alias	Mutable label such as champion	Stable deployment reference	Alias versus stage/version
Compatibility	Schema and runtime expectations	Supports safe upgrades	Backward compatibility test

5. Algorithm / Working Process

Train and evaluate a candidate; serialize the complete inference pipeline; compute its digest; write signature and lineage; upload to a write-once location; register a version; pass security/quality checks; assign a deployment alias; record deployments; retain the previous version for rollback.

6. Mathematical Foundation

A cryptographic digest maps bytes B to a fixed identifier:

```
h = SHA256(B)
P(collision) is negligible for operational version identity
```

Semantic versioning can communicate interface intent:

```
MAJOR.MINOR.PATCH
MAJOR: incompatible signature/behavior contract
MINOR: compatible capability change
PATCH: compatible fix
```

Metrics are metadata, not identity: two byte-different models can have the same score.

7. Practical Implementation

```
import hashlib, json
from pathlib import Path
import joblib

def package_model(pipeline, signature: dict, lineage: dict, out="model_bundle"):
    root = Path(out)
    root.mkdir(exist_ok=False) # refuse accidental overwrite
    model_path = root / "model.joblib"
    joblib.dump(pipeline, model_path)
    digest = hashlib.sha256(model_path.read_bytes()).hexdigest()
    metadata = {"sha256": digest, "signature": signature, "lineage": lineage}
    (root / "metadata.json").write_text(json.dumps(metadata, indent=2))
```

```
    return digest

# package_model(fitted_pipeline,
#   {"inputs": {"age": "float64"}, "output": "probability"},
#   {"run_id": "123", "data_version": "sha256:abc", "git": "deadbeef"})
```

8. Code Explanation

The bundle directory is created with overwrite protection. The fitted preprocessing-and-model pipeline is serialized together, then hashed. Metadata makes the expected interface and provenance explicit. Do not load untrusted pickle/joblib files because deserialization may execute code.

9. Training / Evaluation

Version the candidate before final validation so test evidence points to exact bytes. Validate signature, representative examples, full metrics, slices, calibration, latency, memory, and dependency compatibility. Compare a candidate to the current champion with explicit acceptance thresholds.

10. Complexity and Cost

Hashing is $O(B)$ in artifact bytes. Storage grows with checkpoints and large foundation models; use retention tiers and deduplication where supported. Quantized or sharded variants should be separate versions linked to a common source model.

11. Common Use Cases

Rollback, regulated audit, edge releases, model-format migration, champion/challenger comparisons, fine-tuned LLM adapters, and serving multiple customer-specific models.

12. Common Mistakes

Overwriting "latest," versioning weights without tokenizer/preprocessor, using score as identity, mutable dependencies, missing signature, untrusted pickle loading, and deleting the rollback artifact too early.

13. Edge Cases / Limitations

Nondeterministic builds may produce different hashes from equivalent training. Very large models require manifests of shards. External feature services can change behavior despite a fixed model. Legal deletion requests can conflict with retention; governance must define exceptions.

14. Variations

Sequential registry versions are human-friendly; content-addressed versions guarantee identity; semantic versions describe compatibility; Git LFS suits moderate binaries; OCI images can package runtime plus model. Know content hashes and aliases for interviews.

15. Related Topics

Experiment tracking explains how an artifact was produced. A model registry adds lifecycle controls. Docker versions runtime, DVC versions data, and CI/CD validates/promotes the version.

16. Interview Questions

1. **What belongs in a model version?** Model, preprocessing/tokenizer, signature, lineage, dependencies, metrics, and license/security metadata.
2. **Why immutable?** Reproduction and rollback require a stable referent.
3. **Hash versus semantic version?** Hash proves content identity; semantic version communicates compatibility.
4. **Alias versus version?** Alias moves; version does not.
5. **Can Git store models?** Small ones, but object stores/registries suit large binaries.
6. **How test compatibility?** Contract tests over schema, dtypes, shapes, representative requests, and outputs.
7. **Why package preprocessing?** Training-serving skew otherwise changes the function.
8. **How roll back?** Redirect traffic/alias to a retained compatible version and verify health.
9. **How version an LLM system?** Base model, adapter, tokenizer, prompt, retrieval corpus/index, evaluator, and runtime.
10. **Security concern?** Unsafe deserialization and artifact tampering; sign artifacts and restrict sources.

17. Practice Tasks

- Hash and package a sklearn pipeline.
- Add signature validation for sample JSON.
- Compare artifact and container digests.
- Debug a tokenizer/model version mismatch.
- Implement champion alias rollback.

18. Project Ideas

Project	Description	Stack/dataset	Resume value
Versioned classifier	Immutable bundles and aliases	sklearn, MLflow; Iris/Adult	Core governance
Edge model fleet	Track FP32/ONNX/INT8 variants	PyTorch, ONNX; CIFAR-10	Deployment optimization
LLM adapter catalog	Version base model and LoRA adapters	Hugging Face; small text set	GenAI lineage

19. Quick Revision

Key idea: a deployable model version is immutable bytes plus interface and lineage. Main operation: SHA-256 over artifact bytes. Metrics: candidate quality and compatibility. Trap: moving filenames. Interview one-liner: "Aliases identify intent; immutable versions identify reality."

20. Final Cheat Sheet

Item	Summary
Input/output	Trained pipeline → immutable identified bundle
Main steps	Serialize, sign/hash, describe, store, register, promote
Key metadata	Signature, run/data/code/environment, metrics
Pros/cons	Rollback/audit / storage and governance
Best use	Every production model

Data Versioning

1. Overview

Data versioning identifies the exact data used by a pipeline while storing large bytes outside ordinary Git. It captures raw snapshots, labels, schemas, transformations, and split definitions. Without it, reproducing a model from code and parameters is impossible.

2. Intuition

Git records a recipe, but data versioning records which ingredients were used. "customers.csv" is an address; "SHA-256 abc..." or an immutable table snapshot is an identity.

3. Prerequisites

Files/object storage, databases, Git, hashing, schemas, ETL pipelines, partitioning, and train/validation/test methodology.

4. Core Concepts

Concept	Meaning / example	Why it matters	Interview angle
Snapshot	Immutable view at a time/version	Repeatable training	Snapshot versus backup
Content hash	Identity from bytes	Detects change	Cost for huge datasets
Manifest	Paths, hashes, sizes, schema	Versions a collection	How handle partitions?
Lineage DAG	Raw → cleaned → features → split	Root-cause tracing	Which transformations record?
Schema contract	Types, nullability, ranges	Stops incompatible data	Schema versus distribution validation
Point-in-time correctness	Features only from information available then	Prevents leakage	Explain offline/online skew

5. Algorithm / Working Process

Ingest raw data into an immutable partition; validate schema and quality; compute checksums or record warehouse snapshot IDs; create a manifest; commit the small manifest and pipeline code; materialize features deterministically; version labels and split membership; link the final dataset ID to each training run.

6. Mathematical Foundation

For file chunks $c_1 \dots c_k$, a Merkle-style dataset identity can be derived from ordered hashes:

```
h_i = SHA256(c_i)
H_dataset = SHA256(h_1 || h_2 || ... || h_k || schema_hash)
```

Distribution validation may compare category frequencies with total variation distance:

$$\text{TVD}(P, Q) = 0.5 * \sum_j |P_j - Q_j|$$

Point-in-time joins require:

```
feature_event_time <= prediction_time
```

and often select the latest eligible record per entity.

7. Practical Implementation

```
from pathlib import Path
import hashlib, json

def file_hash(path: Path) -> str:
    h = hashlib.sha256()
    with path.open("rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b''):
            h.update(chunk)
    return h.hexdigest()

def make_manifest(folder: str, output="data-manifest.json"):
    root = Path(folder)
    files = [
        {"path": str(p.relative_to(root)), "bytes": p.stat().st_size,
         "sha256": file_hash(p)}
        for p in sorted(root.rglob("*")) if p.is_file()
    ]
    manifest = {"root": root.name, "files": files}
```

```
Path(output).write_text(json.dumps(manifest, indent=2), encoding="utf-8")
return manifest
```

8. Code Explanation

Files are sorted so manifest ordering is deterministic. Streaming chunks avoids loading large files into memory. The manifest is small enough for Git while actual data can stay in controlled object storage. Production manifests also include schema, partitions, source snapshot IDs, access policy, and transformation lineage.

9. Training / Evaluation

Persist the exact split membership, label cutoff, sampling logic, and preprocessing version. Validate uniqueness, nulls, ranges, referential integrity, class balance, leakage, and important slices before training. Evaluation datasets should be immutable and access-controlled to prevent accidental tuning.

10. Complexity and Cost

Full hashing is $O(\text{total bytes})$ and can be I/O-bound. Incremental manifests hash only changed partitions. Snapshot storage may be cheap with copy-on-write tables but retention still costs money. Network transfer often dominates compute, so train close to data.

11. Common Use Cases

Reproducible training, changing labels, annotation workflows, warehouse time travel, regulated datasets, feature backfills, and collaborative research.

12. Common Mistakes

Versioning only raw data, mutable object keys, missing schema/split versions, random resplitting, leakage in point-in-time joins, committing sensitive data to Git, and treating a timestamp as proof of content.

13. Edge Cases / Limitations

Streaming data may not have a finite snapshot; external licensed datasets may be non-archivable; legal erasure can intentionally break reproduction; nondeterministic queries can change row order/results; huge datasets make full materialization expensive.

14. Variations

DVC versions file-based datasets; lakehouse formats provide table snapshots/time travel; Git LFS stores moderate files; data catalogs focus discovery/governance; event logs enable replay. Placements should compare file manifests with database snapshots.

15. Related Topics

Feature stores add reusable point-in-time feature computation. Experiment tracking links a dataset version to a run. Drift detection compares production data with a reference version. Reproducibility requires both data and environment identity.

16. Interview Questions

1. **Why not put large data in Git?** Repository performance, duplication, access control, and storage are poor.
2. **Path versus version?** A path may mutate; a version is immutable.
3. **What is a manifest?** Metadata listing dataset components and their identities.
4. **How version warehouse data?** Snapshot/table version, query text, parameters, and source snapshot IDs.
5. **What is point-in-time correctness?** No feature may use information later than the prediction timestamp.
6. **Why version splits?** Recreated random splits change evaluation.
7. **Hashing limitation?** It proves byte identity, not semantic quality.
8. **How handle PII?** Controlled remote storage, encryption, least privilege, and metadata-only Git history.
9. **What about corrections?** Create a new version; do not mutate history.
10. **Schema drift versus data drift?** Schema drift changes structure; data drift changes distributions within a valid structure.

17. Practice Tasks

- Generate and verify a file manifest.
- Version a dataset and fixed temporal split.
- Reproduce an old model after changing current data.
- Debug a point-in-time leakage query.
- Add incremental partition hashing.

18. Project Ideas

Project	Description	Stack/dataset	Resume value
Data lineage pipeline	Raw-to-feature manifests	DVC, Pandas; NYC Taxi	Reproducibility
Label audit system	Version corrections and annotators	Parquet, DuckDB; text labels	Data-centric ML
Time-travel benchmark	Compare models over snapshots	lakehouse/DVC; retail data	Temporal rigor

19. Quick Revision

Key idea: version content, schema, transformations, labels, and splits. Main formula: deterministic hash of ordered component hashes. Metrics: data quality and distribution checks. Trap: mutable paths and future-aware joins. Interview one-liner: "Code tells me how; a data version tells me exactly what."

20. Final Cheat Sheet

Item	Summary
Input/output	Dataset bytes/table state → immutable ID and manifest
Steps	Snapshot, validate, hash/version, manifest, store, link

Item	Summary
Controls	Schema, source, cutoff, split, lineage, permissions
Pros/cons	Reproduction/audit / storage and I/O
Best use	Any retrained or regulated ML system

Model Registry

1. Overview

A model registry is a governed catalog of deployable model versions. It stores metadata, signatures, lineage, evaluation evidence, aliases/stages, approvals, and deployment history. Unlike experiment tracking, it represents selected release candidates and their lifecycle.

2. Intuition

Experiment tracking is the research notebook; the registry is the controlled warehouse and release desk. Thousands of runs may exist, but only reviewed artifacts become candidates, champions, or archived versions.

3. Prerequisites

Model/artifact versioning, experiment tracking, deployment environments, access control, CI/CD gates, signatures, and rollback.

4. Core Concepts

Concept	Meaning	Why it matters	Interview angle
Registered model	Named product/model family	Stable ownership boundary	Model name versus run
Version	Immutable artifact reference	Exact deployment	Can it be overwritten?
Alias/stage	champion, challenger, candidate	Decouples deployment config	Why prefer aliases?
Signature	Typed input/output contract	Serving compatibility	Enforcement strategy
Approval	Human or policy authorization	Separation of duties	Automated versus manual gate
Lineage	Run/data/code/environment	Audit and reproduction	Required metadata
Webhook/event	Notification of state change	Triggers delivery	Avoid deployment loops

5. Algorithm / Working Process

Select a successful tracked run; register its immutable artifact and signature; attach metrics, lineage, documentation, owner, and risk level; run validation/security checks; approve or reject; assign

candidate/champion alias atomically; deploy by resolved version; record environment and traffic; archive only after retention requirements pass.

6. Mathematical Foundation

Promotion can be a constraint decision:

```
promote = 1[
    quality_candidate >= quality_champion + delta
    and latency_p99 <= L
    and fairness_gap <= G
    and critical_vulnerabilities = 0
]
```

For uncertainty-aware gates, require a confidence bound:

```
lower_bound(metric_candidate - metric_champion) > delta
```

7. Practical Implementation

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Candidate:
    version: str
    auc: float
    p99_ms: float
    fairness_gap: float
    critical_vulnerabilities: int

def eligible(c: Candidate, champion_auc: float) -> bool:
    return (
        c.auc >= champion_auc + 0.005
        and c.p99_ms <= 100
        and c.fairness_gap <= 0.05
        and c.critical_vulnerabilities == 0
    )

assert eligible(Candidate("12", .906, 80, .03, 0), .900)
assert not eligible(Candidate("13", .920, 150, .03, 0), .900)
```

8. Code Explanation

The candidate record is immutable and the gate combines quality, operational, fairness, and security constraints. Assertions provide a minimal executable policy check. Real registries retain each check result and approval identity, then update aliases transactionally.

9. Training / Evaluation

Only register complete inference bundles. Promotion evidence should include held-out and slice metrics, uncertainty, schema compatibility, calibration, robustness, load tests, security scan, model card, and comparison with the current champion. Re-evaluate old artifacts when evaluator policy changes.

10. Complexity and Cost

Registry metadata is small; artifacts dominate storage. Governance adds review latency but reduces incident and audit cost. Resolving an alias should be fast and cached, while deployment should pin the resolved immutable version rather than repeatedly fetching a moving alias.

11. Common Use Cases

Champion/challenger management, regulated approvals, multi-environment promotion, rollback, edge fleet releases, foundation-model/adaptor catalogs, and cross-team discovery.

12. Common Mistakes

Using the registry as raw experiment storage, mutable versions, promoting on one metric, deploying a moving alias without recording resolution, missing ownership, no retention rule, and granting every trainer production promotion rights.

13. Edge Cases / Limitations

One model name may hide per-region/customer variants; ensembles need dependency manifests; external APIs have provider versions outside the registry; a registry outage must not break already-running serving; deletion may be restricted by audit policy.

14. Variations

Stage-based workflows use Staging/Production/Archived; alias-based workflows use mutable semantic labels; GitOps stores desired deployment references in Git; federated registries serve multiple regions. Alias-based promotion is increasingly flexible and interview-relevant.

15. Related Topics

MLflow provides tracking and registry features. CI/CD enforces gates, model serving consumes approved versions, monitoring maps incidents back to deployed versions, and model versioning supplies immutable identity.

16. Interview Questions

1. **Registry versus tracker?** Tracker contains all runs; registry governs selected deployable artifacts.
2. **What is a model alias?** A movable name resolving to an immutable version.
3. **Why pin resolved versions?** To know exactly what is running despite alias movement.
4. **What metadata is essential?** Artifact URI/digest, signature, lineage, metrics, owner, approvals, and status.
5. **How promote safely?** Policy gates, approval, atomic alias change, guarded rollout, and rollback.

6. **What should deployment record?** Version/digest, environment, configuration, start/end time, traffic, and actor.
7. **How handle ensembles?** Register a manifest pinning every component and combination logic.
8. **Can registry approval replace testing?** No; it records decisions and evidence.
9. **How secure it?** RBAC, signed artifacts, audit logs, encryption, and separation of duties.
10. **How retire versions?** Remove traffic/aliases, observe retention, archive metadata, then delete eligible artifacts.

17. Practice Tasks

- Define a promotion policy and test boundary values.
- Register two candidates and atomically move a champion alias.
- Reconstruct deployment history from audit events.
- Debug an alias/version mismatch.
- Add an ensemble dependency manifest.

18. Project Ideas

Project	Description	Stack/dataset	Resume value
Governed registry	Quality gates and approvals	MLflow; Adult	Production governance
Multi-model catalog	Search signatures and owners	FastAPI, SQLite, object store	Platform engineering
LLM release registry	Base/adaptor/prompt/evaluator lineage	Hugging Face, MLflow	LLMOps

19. Quick Revision

Key idea: the registry governs deployable immutable versions. Main logic: promote only if all policy constraints pass. Metrics: quality, slices, latency, security, cost. Trap: confusing a run artifact with an approved release. Interview one-liner: "Tracking explains experiments; the registry controls releases."

20. Final Cheat Sheet

Item	Summary
Input/output	Validated artifact → governed release version/alias
Steps	Register, document, validate, approve, alias, deploy, archive
Controls	Signature, RBAC, gates, audit, retention
Pros/cons	Discovery and safe promotion / governance overhead
Best use	Teams with repeated production deployments

Model Serving

1. Overview

Model serving makes a trained model available to consumers through a stable interface. It loads an artifact, validates and transforms inputs, runs inference, transforms outputs, and meets latency, throughput, availability, cost, and security targets. Serving may be online, batch, streaming, edge, or embedded.

2. Intuition

Training produces a chef; serving runs the restaurant. The model is only one station—requests also need admission control, preprocessing, scheduling, postprocessing, observability, and fallbacks.

3. Prerequisites

Inference, serialization, APIs, processes/threads, CPU/GPU memory, containers, queues, load balancing, metrics, and model signatures.

4. Core Concepts

Concept	Meaning and importance	Interview angle
Serving contract	Typed request/response and error behavior	Prevent training-serving mismatch
Runtime	Python, ONNX Runtime, TensorRT, vLLM, etc.	Choose for model/hardware
Concurrency	Simultaneous in-flight work	Threads versus processes
Batching	Combine inputs into one accelerator call	Throughput-latency tradeoff
Autoscaling	Change replicas using load signals	Why CPU may be wrong for GPUs
Warm-up	Initialize kernels/caches before readiness	Cold-start mitigation
Fallback	Rule, cached result, or prior model	Graceful degradation

5. Algorithm / Working Process

Resolve and verify an approved artifact; load model and preprocessing once; warm it; expose liveness/readiness; accept a request; authenticate, validate, and rate-limit; fetch features; preprocess; schedule/batch; infer under a timeout; postprocess; log a correlation ID and metrics; return or fall back. Graceful shutdown stops new traffic and drains in-flight work.

6. Mathematical Foundation

Throughput is λ requests/second and latency is response time. Little's Law gives:

$$\text{average_concurrency} = \text{throughput} * \text{average_latency}$$

Utilization ρ near 1 causes queueing delay to rise sharply. For dynamic batching:

```
latency_total = wait_for_batch + preprocess + inference(batch) + postprocess
throughput = batch_size / inference_time(batch)
```

An SLO may require $P(\text{latency} \leq 100 \text{ ms}) \geq 0.99$ and error rate ≤ 0.001 .

7. Practical Implementation

```
import numpy as np

class Predictor:
    def __init__(self, pipeline):
        self.pipeline = pipeline
        self.version = "sha256:example"

    def predict(self, rows: list[dict]) -> dict:
        if not rows:
            raise ValueError("at least one row is required")
        probabilities = self.pipeline.predict_proba(rows)[: , 1]
        return {
            "model_version": self.version,
            "predictions": np.asarray(probabilities).tolist(),
        }
```

8. Code Explanation

The pipeline is loaded once and owns preprocessing plus inference. Batch-shaped input works for both one and many records. The response exposes the model version for traceability. The transport layer should translate validation/timeouts into stable error responses.

9. Training / Evaluation

Export a complete pipeline and golden examples. Before traffic, test numerical parity with the training runtime, input contracts, calibration, batch shapes, malformed requests, concurrency, memory, warm/cold latency, sustained load, and failure recovery.

10. Complexity and Cost

For model compute $C(x)$, inference is $O(C)$; memory includes weights, runtime, activations, caches, and per-worker duplication. GPUs improve throughput only when sufficiently utilized. Replicas reduce queueing but increase idle cost. Quantization, batching, caching, and compiled runtimes trade accuracy/freshness/complexity for cost.

11. Common Use Cases

Fraud authorization, search ranking, recommendations, document classification, image inspection, LLM generation, nightly scoring, and on-device prediction.

12. Common Mistakes

Loading per request, separate training/serving preprocessing, unbounded queues, no timeouts, reporting averages only, logging sensitive inputs, scaling GPU servers on CPU, and making readiness pass before warm-up.

13. Edge Cases / Limitations

Huge requests can exhaust memory; variable LLM sequence lengths cause head-of-line blocking; feature-service failure dominates latency; nondeterministic runtimes complicate parity; multi-tenant models need isolation; stale caches can violate correctness.

14. Variations

Synchronous serving returns immediately; asynchronous serving returns a job ID; serverless suits bursty lightweight models; dedicated GPU serving suits steady heavy load; edge serving minimizes network dependence. All are important for system-design interviews.

15. Related Topics

REST is a transport; online and batch are timing modes; distributed serving scales the runtime; Kubernetes orchestrates replicas; monitoring measures service/model behavior; compression reduces serving cost.

16. Interview Questions

1. **Training versus serving?** Training optimizes parameters; serving executes a fixed artifact under operational constraints.
2. **Why load once?** Loading per request adds latency and memory churn.
3. **Liveness versus readiness?** Alive process versus safe to receive traffic.
4. **Why batch?** Better vectorization/GPU utilization, at the cost of waiting.
5. **How handle overload?** Bounded queues, admission control, timeouts, autoscaling, and fallback.
6. **What is cold start?** Artifact/runtime initialization before useful inference.
7. **What identifies a response?** Request/correlation ID plus immutable model version.
8. **How check parity?** Golden inputs and tolerance-based output comparison.
9. **What scale signal for GPUs?** Queue depth, in-flight requests, batch fill, or accelerator utilization.
10. **How roll back?** Route to a retained compatible artifact and verify SLOs.

17. Practice Tasks

Build a reusable predictor; benchmark batch sizes; load-test p50/p95/p99; inject a feature-store timeout; add a fallback and version field.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Fraud scorer	Low-latency calibrated scoring	FastAPI, sklearn; IEEE-CIS subset	Online design

Project	What it does	Stack/dataset	Resume value
Image batcher	Dynamically batches images on GPU	PyTorch; CIFAR-10	Throughput tuning
LLM endpoint	Streams tokens with admission control	vLLM/HF; small instruct model	GenAI serving

19. Quick Revision

Key idea: serving is a constrained inference system, not a predict call. Main formula: concurrency = throughput × latency. Metrics: p50/p95/p99, QPS, errors, saturation, cost, model behavior. Trap: unbounded queues. Interview one-liner: “I pin an immutable artifact, validate the contract, warm it, bound work, observe every version, and keep a fallback.”

20. Final Cheat Sheet

Definition: reliable access to model inference. Input/output: validated features to versioned predictions. Steps: load, warm, admit, preprocess, infer, postprocess, observe. Knobs: replicas, workers, batch size/wait, timeout. Pros: reusable centralized inference; cons: latency, cost, operational complexity.

REST API Deployment

1. Overview

REST API deployment exposes inference over HTTP resources and methods, commonly JSON over POST. It suits interoperable synchronous services, but production deployment also requires validation, authentication, versioning, timeouts, rate limits, observability, and safe rollout.

2. Intuition

REST is the reception desk around the model. It checks the form, sends accepted work to the predictor, and returns a documented envelope—without exposing how the model is implemented.

3. Prerequisites

HTTP, JSON, status codes, schemas, FastAPI/Flask concepts, model serialization, containers, networking, and security basics.

4. Core Concepts

Concept	Meaning / example	Interview angle
Contract	POST /v1/predict schema	How evolve safely?
Idempotency	Same keyed request does not duplicate side effects	Important for retries
Status codes	400/422 client, 401/403 auth, 429 limit, 5xx server	Do not return 200 with error

Concept	Meaning / example	Interview angle
API version	Interface version, not model version	Distinguish both
Timeout/retry	Bound waiting; retry only safe transient work	Retry storms
Authentication	API key/OAuth/mTLS	Authn versus authz

5. Algorithm / Working Process

Define an OpenAPI contract; load/warm the model at process start; expose health routes; authenticate and cap body size; parse and validate; attach request ID; call predictor with timeout; map known errors; emit structured metrics/logs; return model version; deploy behind TLS/load balancer with canary and rollback.

6. Mathematical Foundation

Availability and latency compose across serial dependencies:

```
A_total approximately product_i A_i
L_total approximately sum_i L_i + queueing
```

If each retry succeeds independently with transient failure probability p , retries reduce failure probability to $p^{(k+1)}$, but increase load. Use exponential backoff with jitter and a retry budget.

7. Practical Implementation

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

app = FastAPI(title="Risk API", version="1.0")
model = None # assign a verified pipeline during startup

class Request(BaseModel):
    age: float = Field(ge=0, le=120)
    income: float = Field(ge=0)

@app.get("/health/ready")
def ready():
    if model is None:
        raise HTTPException(503, "model not loaded")
    return {"ready": True}

@app.post("/v1/predict")
def predict(item: Request):
    probability = float(model.predict_proba([item.model_dump()])[0, 1])
    return {"probability": probability, "model_version": "12"}
```

8. Code Explanation

Pydantic rejects invalid values before inference. Readiness fails until initialization finishes. The API version describes the HTTP contract, while model_version identifies learned behavior. Deployment should use multiple worker processes only after accounting for duplicated model memory.

9. Training / Evaluation

Besides offline model tests, run schema contract tests, golden predictions, malformed/oversized input tests, auth tests, dependency failures, load/soak tests, and canary comparison. Measure client-observed latency, not only model time.

10. Complexity and Cost

JSON parsing and network hops add overhead. Payload size is $O(\text{number of features})$; inference follows model complexity. Binary formats/gRPC can help high-throughput internal traffic. Worker count is bounded by memory; each process may copy weights.

11. Common Use Cases

Web/mobile predictions, internal microservices, document scoring, moderation, recommendations, and prototype-to-production inference.

12. Common Mistakes

GET requests with sensitive features, no schema bounds, synchronous blocking I/O in async handlers, global mutable request state, exposing stack traces, unlimited payloads, blind retries, and conflating API/model versions.

13. Edge Cases / Limitations

Long LLM generation benefits from streaming/asynchronous jobs; huge images need object URLs or multipart upload; strict real-time systems may need gRPC; REST does not guarantee exactly-once delivery; client disconnects may leave computation running.

14. Variations

REST/JSON maximizes compatibility; gRPC/Protobuf improves typed internal performance; WebSockets/SSE stream updates; async job APIs suit long inference; GraphQL is rarely needed for a fixed prediction command.

15. Related Topics

FastAPI implements the transport, Docker packages it, Kubernetes operates it, model serving owns inference, logging/monitoring expose behavior, and CI/CD validates/release it.

16. Interview Questions

1. **Why POST?** Prediction bodies can be large/sensitive and are not resource retrieval URLs.
2. **422 versus 500?** Invalid client schema versus unexpected server failure.
3. **API versus model version?** Contract compatibility versus artifact identity.
4. **How retry?** Only idempotent/transient failures, with backoff, jitter, and budget.
5. **Why return model version?** Traceability and incident analysis.

6. **How protect endpoint?** TLS, authn/authz, rate/body limits, validation, secret management.
7. **Async handler always faster?** No; it helps waiting I/O, not CPU-bound inference.
8. **How handle long jobs?** Return 202 plus job ID, then poll/callback.
9. **What health checks?** Cheap liveness and dependency/model-aware readiness.
10. **What do you load-test?** Tail latency, throughput, errors, saturation, memory, and recovery.

17. Practice Tasks

Create a validated endpoint; add contract tests; reject oversized batches; benchmark workers; add request IDs and a 503 readiness state.

18. Project Ideas

Project	Description	Stack/dataset	Resume value
Credit risk API	Versioned probability service	FastAPI, sklearn; German Credit	API fundamentals
Async document API	Job-based NLP inference	FastAPI, Redis; AG News	Queue design
Image moderation API	Upload/URI scoring	FastAPI, PyTorch; CIFAR subset	Media contracts

19. Quick Revision

REST wraps inference in a versioned HTTP contract. Main operational math is serial latency/availability. Metrics: request rate, status codes, tail latency, saturation. Trap: returning 200 for errors or retrying overload. One-liner: "Validate at the boundary, separate API from model version, bound every resource, and make retries safe."

20. Final Cheat Sheet

Input/output: JSON features to JSON prediction. Steps: authenticate, validate, infer, observe, respond. Knobs: timeout, batch cap, workers, rate limit. Pros: universal and debuggable; cons: serialization overhead and poor fit for long streaming work.

Batch Inference

1. Overview

Batch inference scores a finite dataset on a schedule or trigger and writes results to durable storage. It favors throughput, cost efficiency, and repeatability over per-request latency. Examples include nightly recommendations, weekly churn lists, and backfilling embeddings.

2. Intuition

Instead of cooking each order immediately, batch inference prepares tomorrow's meals together. Vectorized reads and large accelerator batches reduce overhead, while outputs can be checked before consumers use

them.

3. Prerequisites

Dataframes/SQL, partitioned storage, schedulers, idempotency, model loading, distributed data processing, and data-quality checks.

4. Core Concepts

Concept	Meaning / importance	Interview angle
Partition	Bounded shard by date/entity	Parallelism and recovery
Idempotency	Rerun creates same logical result	How avoid duplicate rows?
Checkpoint	Completed partitions/watermark	Resume failures
Snapshot consistency	One model and feature cutoff	Prevent mixed versions
Backfill	Recompute historical partitions	Resource isolation
Output contract	IDs, scores, version, timestamps	Downstream traceability

5. Algorithm / Working Process

Pin model/data/feature versions and cutoff; discover input partitions; validate schema; load model once per worker; read a bounded chunk; transform and vectorize; predict; attach entity ID, event time, model version, and run ID; write to a staging partition; validate counts/ranges; atomically publish; record lineage and retry failed partitions.

6. Mathematical Foundation

For N rows, batch size b, and per-batch time T(b):

```
number_of_batches = ceil(N / b)
throughput = b / T(b)
total_time approximately ceil(N/b) * T(b) / parallel_workers + I/O
```

Cost per prediction is total compute/storage cost divided by successful unique outputs. Completeness is scored_rows / eligible_rows.

7. Practical Implementation

```
import pandas as pd

def score_file(model, input_csv, output_parquet, model_version,
               chunk_size=10_000):
    pieces = []
    for chunk in pd.read_csv(input_csv, chunksize=chunk_size):
        ids = chunk.pop("entity_id")
```

```
scores = model.predict_proba(chunk)[: , 1]
pieces.append(pd.DataFrame({
    "entity_id": ids,
    "score": scores,
    "model_version": model_version,
}))
result = pd.concat(pieces, ignore_index=True)
assert result["entity_id"].is_unique
result.to_parquet(output_parquet, index=False)
```

8. Code Explanation

Chunking bounds input memory, although accumulating pieces still holds outputs; production jobs write each chunk to staging partitions. Entity IDs are preserved, version metadata accompanies every prediction, and uniqueness catches accidental duplication.

9. Training / Evaluation

Evaluate historical temporal backtests and compare batch feature computation with training. At runtime validate eligible/scored counts, duplicates, missingness, score ranges, slice distributions, and a sampled golden set. When labels arrive, join by entity and prediction timestamp/version.

10. Complexity and Cost

Compute is $O(N \text{ times model cost})$; memory is $O(\text{chunk size plus model})$. Larger chunks improve throughput until memory spills. Distributed systems add shuffle and startup costs; partition-local transformations avoid shuffles. Spot/preemptible compute can lower cost if checkpointing works.

11. Common Use Cases

Marketing lists, demand forecasts, loan reviews, recommendation candidates, periodic risk scoring, embedding/index generation, and model backfills.

12. Common Mistakes

Loading model per row, overwriting outputs before validation, mixing model versions, non-idempotent append retries, missing IDs/timestamps, reading all data into memory, and backfills starving daily jobs.

13. Edge Cases / Limitations

Late data changes closed partitions; input skew creates stragglers; outputs can become stale between schedules; a partial publish misleads consumers; historical features may not be reconstructable; very small jobs do not justify distributed frameworks.

14. Variations

Micro-batching scores short windows; streaming continuously processes events; map-only jobs suit independent rows; distributed GPU batch scoring suits foundation models. Choose the simplest method meeting freshness and volume.

15. Related Topics

Online inference optimizes immediate latency; DVC/data versioning pins snapshots; Airflow/Kubeflow schedule jobs; Ray/Spark distribute work; monitoring checks freshness and completeness.

16. Interview Questions

1. **Batch versus online?** Scheduled throughput/durable outputs versus immediate response.
2. **How make reruns safe?** Deterministic keys, staging, partition replacement/upsert, and atomic publish.
3. **What metadata per prediction?** Entity/event time, score, model version, run ID, feature cutoff.
4. **How recover?** Checkpoint completed partitions and retry only failed units.
5. **How avoid OOM?** Chunked reads/writes and tuned batch size.
6. **How detect partial output?** Expected counts, completion manifest, and atomic publication.
7. **What about late data?** Watermarks, correction windows, and explicit backfills.
8. **When distribute?** When measured single-node time/capacity misses SLA.
9. **How prevent skew?** Better partition keys, split hot keys, dynamic scheduling.
10. **How evaluate later?** Point-in-time join matured labels to immutable predictions.

17. Practice Tasks

Chunk-score a CSV; make reruns idempotent; simulate failure/resume; validate row counts; benchmark local versus parallel execution.

18. Project Ideas

Project	Description	Stack/dataset	Resume value
Nightly churn scores	Partitioned publish with audit manifest	Pandas, DVC; Telco	Reliable batch design
Embedding backfill	Resumeable document embedding	Ray, HF; Wikipedia subset	Distributed inference
Demand forecast job	Rolling temporal predictions	Prefect/Kubeflow; M5	Scheduling and time series

19. Quick Revision

Key idea: pin a snapshot, score partitions, validate, then atomically publish. Formula: total time $\approx$ batches $\times$ batch time / workers + I/O. Metrics: throughput, completeness, freshness, duplicates, cost. Trap: append-on-retry duplicates.

20. Final Cheat Sheet

Input/output: versioned dataset to durable prediction table. Knobs: chunk/batch size, workers, partitioning, retry policy. Pros: cheap, auditable, high throughput; cons: stale results and pipeline delay. Best for decisions tolerant of scheduled freshness.

Online Inference

1. Overview

Online inference computes predictions in response to live events or requests, normally within milliseconds to seconds. It powers fraud authorization, ranking, personalization, and interactive assistants. Correctness includes freshness, latency, availability, and behavior under overload.

2. Intuition

It is an emergency-room triage desk: each case arrives unpredictably, must use current information, and needs a bounded response time. A slightly less accurate but reliable fast model can create more value than a slow offline winner.

3. Prerequisites

Serving/API concepts, latency percentiles, feature stores/caches, concurrency, autoscaling, fallbacks, model calibration, and distributed-system failure modes.

4. Core Concepts

Concept	Meaning / importance	Interview angle
Tail latency	p95/p99 captures slow users	Why average is insufficient
Fresh features	Latest causally valid state	Offline-online consistency
Admission control	Reject/degrade excess load	Protect latency
Deadline propagation	One end-to-end time budget	Allocate dependency budgets
Fallback	Cached/rule/prior-model result	Availability over ideal output
Calibration/threshold	Convert score into action	Business cost tradeoff

5. Algorithm / Working Process

Receive an authenticated event with a deadline; validate/deduplicate; fetch or compute point-in-time features; apply the exact transform; run a warm model; calibrate/rank/threshold; log version and minimal features safely; return before deadline. On dependency failure or overload, use a defined fallback or fail closed/open according to risk.

6. Mathematical Foundation

For binary action with false-positive cost C_{FP} and false-negative cost C_{FN} , an ideal calibrated threshold under simple assumptions is:

$$t = C_{FP} / (C_{FP} + C_{FN})$$

End-to-end budget:

```
L_total = L_gateway + L_feature + L_queue + L_model + L_post
P99_total is not generally the sum of component P99 values
```

Capacity should keep utilization below saturation and account for burst arrival rates.

7. Practical Implementation

```
import time

def decide(predictor, feature_client, entity_id, deadline_ms=100):
    started = time.perf_counter()
    try:
        features = feature_client.get(entity_id, timeout_ms=30)
        score = predictor.predict(features)
        elapsed_ms = (time.perf_counter() - started) * 1000
        if elapsed_ms > deadline_ms:
            raise TimeoutError("deadline exceeded")
        return {"score": score, "source": "model"}
    except (TimeoutError, ConnectionError):
        return {"score": 0.5, "source": "fallback"}
```

8. Code Explanation

The feature call gets only part of the budget. Expected transient failures and deadline overruns use an explicit fallback whose source is observable. Production code propagates cancellation, distinguishes risk policies, and records fallback rates.

9. Training / Evaluation

Use time-aware training and point-in-time features. Validate calibration and thresholds on realistic prevalence, then replay traffic and load-test bursts, cold starts, dependency faults, and payload extremes. Evaluate online business impact through controlled experiments, not request accuracy alone.

10. Complexity and Cost

Per-request time includes feature retrieval plus model complexity. Replicated warm capacity costs more but avoids queueing/cold starts. Caching saves compute at the cost of staleness. GPU batching improves throughput but adds wait time; small CPU models often win at low QPS.

11. Common Use Cases

Payment fraud, ad/search ranking, recommendations, dynamic pricing, content safety, chat assistants, and predictive maintenance alerts.

12. Common Mistakes

Average-only latency, synchronous chains with no deadlines, online feature logic different from training, retrying within a nearly spent budget, no overload policy, silent fallbacks, and thresholds tuned on old prevalence.

13. Edge Cases / Limitations

Cold entities lack history; burst traffic defeats average capacity; hot keys overload one shard; feedback loops change future inputs; strict financial/safety decisions may need deterministic fail-closed behavior; label delays obscure quality.

14. Variations

Request-response scoring, event-stream processing, nearline micro-batches, edge inference, and hybrid precompute-plus-rerank. Nearline is useful when seconds/minutes of freshness meet the product need.

15. Related Topics

Feature stores supply low-latency values; distributed serving scales inference; REST/gRPC transports requests; monitoring and drift track health; A/B and shadow deployments reduce release risk.

16. Interview Questions

1. **Why p99?** Tail failures affect users and reveal saturation.
2. **How divide a deadline?** Reserve time for gateway/postprocessing and cap every dependency.
3. **What is training-serving skew?** Different feature/transformation behavior across environments.
4. **How handle feature outage?** Cached/default features, prior score, fallback model, or risk-specific failure.
5. **Why not retry everything?** Retries consume remaining deadline and amplify overload.
6. **How scale bursty traffic?** Headroom, queue/admission control, fast autoscaling, and graceful degradation.
7. **When use a GPU?** When model/batching/QPS yields better measured latency-cost.
8. **How trace decisions?** Request ID, timestamp, model/feature versions, score, action, and safe diagnostics.
9. **What is a hot key?** One entity/partition receives disproportionate load.
10. **How assess quality without labels?** Input/output drift, proxy outcomes, shadow comparisons, then delayed labels.

17. Practice Tasks

Budget component timeouts; simulate bursts; implement a fallback; calibrate an action threshold; diagnose online/offline feature mismatch.

18. Project Ideas

Project	Description	Stack/dataset	Resume value
Live fraud gate	Deadline/fallback scoring	FastAPI, Redis; fraud data	Real-time reliability

Project	Description	Stack/dataset	Resume value
Two-stage recommender	Precomputed candidates + live rerank	Python, feature cache; MovieLens	System design
Streaming sentiment	Event-based scoring and sink	Kafka/Ray; tweets	Stream inference

19. Quick Revision

Key idea: immediate prediction under a deadline and live state. Main formulas: cost-based threshold and component latency budget. Metrics: p99, QPS, errors, timeouts, fallback, saturation, delayed quality. Trap: retries and stale/inconsistent features.

20. Final Cheat Sheet

Input/output: live event to immediate decision. Steps: admit, fetch, transform, infer, decide, observe/fallback. Knobs: deadlines, concurrency, threshold, cache TTL, replicas. Pros: fresh personalization; cons: higher cost and reliability burden.

Monitoring

1. Overview

ML monitoring continuously checks whether a deployed system is available, fast, data-correct, behaviorally stable, useful, fair, and affordable. It spans infrastructure, service, data, model outputs, delayed ground truth, and business outcomes. A dashboard describes; an alert demands an owned action.

2. Intuition

A model is an aircraft in changing weather. Engine gauges (CPU/errors) are necessary, but navigation (business outcome), fuel (cost), sensors (data), and destination error (model quality) determine whether the flight succeeds.

3. Prerequisites

Metrics/logs/traces, SLI/SLO/SLA, time series, percentiles, model evaluation, drift, alerting, privacy, and incident response.

4. Core Concepts

Layer	Examples	Why/interview angle
Infrastructure	CPU/GPU, memory, disk, restarts	Find saturation/resource failure
Service	rate, errors, duration, queue depth	RED method and SLOs
Data quality	schema, nulls, ranges, freshness	Catch broken pipelines

Layer	Examples	Why/interview angle
Model behavior	score/class/ranking distributions	Works without labels, not proof of accuracy
Performance	precision, recall, calibration by slice	Needs matured labels
Business	conversion, loss prevented, user retention	Confounding and experiments
Cost	compute, tokens, cache hit, cost/prediction	Keep system economically viable

5. Algorithm / Working Process

Define owners and SLOs; instrument each deployed version; establish reference windows and important slices; aggregate privacy-safe signals; validate freshness/completeness; compare current and reference behavior; join matured labels point-in-time; alert only on actionable sustained breaches; triage data/service/model/business causes; mitigate, annotate, and review.

6. Mathematical Foundation

```
error_rate = failed_requests / valid_requests
availability = successful_eligible_requests / eligible_requests
error_budget = (1 - 0.999) * 30 days = 43.2 minutes
Brier = (1/n) sum_i (p_i - y_i)^2
```

7. Practical Implementation

```
from collections import Counter

class Window:
    def __init__(self):
        self.counts, self.latencies = Counter(), []

    def observe(self, status, latency_ms):
        self.counts[status] += 1
        self.latencies.append(latency_ms)

    def summary(self):
        xs, n = sorted(self.latencies), len(self.latencies)
        return {
            "requests": n,
            "error_rate": self.counts["error"] / max(n, 1),
            "p99_ms": xs[min(n - 1, int(.99 * n))] if n else None,
        }
```

8. Code Explanation

The accumulator illustrates request count, failure ratio, and tail latency. Production systems use histograms instead of retaining timings, aggregate by bounded labels such as route/version/region, and keep high-cardinality request IDs in logs/traces.

9. Training / Evaluation

Record reference distributions, baseline metrics, calibration, and slices as deployable metadata. Validate alerts with replayed incidents and synthetic failures. When labels mature, evaluate only predictions whose outcome window has closed and compare with the champion/control using uncertainty.

10. Complexity and Cost

Metrics scale with time-series count; unbounded labels create high bills. Logs scale with events/payload. Sample traces, aggregate distributions, and retain raw examples only with privacy controls. Instrumentation must not violate inference SLOs.

11. Common Use Cases

Fraud quality with delayed labels, LLM cost/safety, recommender CTR, feature freshness, GPU saturation, bias slices, and batch completeness.

12. Common Mistakes

Alerting on every fluctuation, no owner/runbook, average latency only, raw PII logs, treating drift as accuracy loss, missing version tags, incomplete label windows, and dashboards without business metrics.

13. Edge Cases / Limitations

Labels may be absent, delayed, censored, or affected by model actions. Seasonality causes expected shifts. Rare harms evade aggregates. Monitoring detects symptoms; causal diagnosis may require experiments.

14. Variations

White-box monitoring instruments internals; black-box monitoring observes endpoints. Real-time alerts suit service failures; scheduled reports suit slow quality changes. LLM monitoring adds groundedness, toxicity, retrieval quality, tokens, and human evaluation.

15. Related Topics

Logging supplies events, drift detection supplies comparisons, A/B tests estimate causal impact, registries map metrics to versions, and CI/CD deploys instrumentation.

16. Interview Questions

1. **What monitor without labels?** Schema, freshness, inputs, outputs, confidence, latency, errors, saturation, and proxies.
2. **Does drift prove degradation?** No; verify labels or business impact.
3. **SLI/SLO/SLA?** Indicator, internal target, external commitment.
4. **Why p99?** Averages hide tail pain and saturation.
5. **What is alert fatigue?** Nonactionable alerts teach responders to ignore them.

6. **How monitor delayed labels?** Mature cohorts plus interim proxies.
7. **Avoid cardinality explosion?** Bounded metric labels; IDs in logs/traces.
8. **What is an error budget?** Allowed unreliability under the SLO.
9. **How monitor bias?** Predefined eligible groups and uncertainty-aware slices.
10. **First incident question?** Which versions, slices, regions, dependencies, and time boundaries changed?

17. Practice Tasks

Create a RED dashboard; define two SLOs; simulate latency/schema incidents; compute mature-cohort F1; write an alert/runbook pair.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Model control room	Service/data/model dashboard	Prometheus, Grafana; synthetic	Observability
Delayed-label monitor	Cohort fraud evaluation	Pandas, Evidently; fraud	Temporal correctness
LLM monitor	Cost, latency, safety, grounding	OpenTelemetry, RAG corpus	LLMOps

19. Quick Revision

Key idea: monitor service, data, behavior, quality, business, fairness, and cost by version/slice. Formula: error budget = $(1 - \text{SLO}) \times \text{window}$. Traps: label leakage, cardinality, noisy alerts. One-liner: "Every alert has an owner, action, version dimension, and sustained threshold."

20. Final Cheat Sheet

Inputs: telemetry, predictions, labels, outcomes. Outputs: dashboards, alerts, incident evidence. Knobs: windows, thresholds, slices, retention. Pros: early detection; cons: cost, noise, incomplete causality.

Drift Detection

1. Overview

Drift detection tests whether production data or relationships differ materially from a reference. Covariate drift changes $P(X)$, label drift changes $P(Y)$, and concept drift changes $P(Y|X)$. Detection is diagnostic; it does not automatically mean retraining will help.

2. Intuition

A weather model trained in summer may see winter inputs (covariate drift); rainy-day frequency may change (label drift); or the same clouds may predict rain differently after moving region (concept drift).

3. Prerequisites

Distributions, hypothesis tests, sampling, histograms, KL/JS divergence, KS/chi-square tests, calibration, temporal validation, and multiple testing.

4. Core Concepts

Concept	Meaning	Interview angle
Reference/current windows	Training/recent populations	Window and seasonality choice
Univariate drift	One feature changes	Misses interactions
Multivariate drift	Joint distribution changes	Classifier two-sample test
Concept drift	Conditional mapping changes	Usually needs labels
Data quality issue	Broken values/schema	Not genuine drift
Practical threshold	Effect size and impact	p-value is insufficient

5. Algorithm / Working Process

Validate schema; choose representative windows; align eligible populations; segment expected cohorts; inspect missing/novel categories; compute feature-appropriate tests/effect sizes; correct repeated tests; inspect outputs/proxies; confirm with labels/business impact; remediate upstream, recalibrate, retrain, or annotate.

6. Mathematical Foundation

```
PSI = sum_i (q_i - p_i) ln(q_i / p_i)
M = (P + Q)/2
JS(P,Q) = 0.5 KL(P||M) + 0.5 KL(Q||M)
KS: D = sup_x |F_ref(x) - F_cur(x)|
```

A domain classifier trained to distinguish reference/current data exposes multivariate drift; AUC near 0.5 means poor separability.

7. Practical Implementation

```
import numpy as np

def psi(reference, current, bins=10, eps=1e-6):
    edges = np.quantile(reference, np.linspace(0, 1, bins + 1))
    edges[0], edges[-1] = -np.inf, np.inf
    p = np.histogram(reference, edges)[0] / len(reference)
    q = np.histogram(current, edges)[0] / len(current)
    p, q = np.clip(p, eps, None), np.clip(q, eps, None)
    return float(np.sum((q - p) * np.log(q / p)))
```

```
assert psi(np.arange(1000), np.arange(1000)) < 1e-12
```

8. Code Explanation

Reference quantiles make balanced baseline bins. Infinite endpoints retain out-of-range values; epsilon prevents division by zero. PSI thresholds are heuristics calibrated to sample size, seasonality, and impact.

9. Training / Evaluation

Save reference distributions by valid slice/version. Backtest detectors on stable periods and known shifts; measure false-alert rate, delay, and relation to degradation. Evaluate retraining on a forward holdout, not the trigger window.

10. Complexity and Cost

Histograms are $O(n+b)$; sorting quantiles is $O(n \log n)$ without sketches. Multivariate methods grow with dimension. Sampling/sketches reduce stream cost. Thousands of tests require false-discovery control.

11. Common Use Cases

Fraud changes, camera/sensor changes, seasonal demand, topic evolution, pipeline breakage, and new populations.

12. Common Mistakes

Calling schema failure drift, p-values without effect size, tiny shifts on huge samples, ignoring seasonality, comparing different populations, means only, and automatic retraining.

13. Edge Cases / Limitations

$P(X)$ can drift harmlessly; concept drift can occur with stable $P(X)$; correlated tests create alert storms; sparse/new categories need smoothing; feedback loops alter observed labels.

14. Variations

PSI/KS/chi-square are univariate; MMD/energy distance compare samples; domain-classifier AUC is multivariate; ADWIN/DDM detect streams; performance drift monitors loss when labels exist.

15. Related Topics

Monitoring operationalizes signals; data versioning supplies references; calibration may fix prior shift; active learning targets new regions; causal analysis separates policy/environment changes.

16. Interview Questions

1. **Three drift types?** $P(X)$, $P(Y)$, and $P(Y|X)$.
2. **Concept drift without labels?** Not reliably in general.
3. **Why not p-value alone?** It scales with sample size and ignores impact.

4. **PSI weakness?** Bin-sensitive heuristic with no universal threshold.
5. **Handle seasonality?** Compare aligned windows or condition on season.
6. **Domain-classifier drift?** Predict whether a row is reference/current.
7. **Why multiple-test correction?** Many features cause false discoveries.
8. **Alert response?** Validate pipeline/population, inspect impact, then remediate.
9. **Recalibrate when?** Ranking is useful but probability mapping/prevalence changed.
10. **Detector metrics?** False alarms, delay, effect size, degradation correlation.

17. Practice Tasks

Implement PSI/KS; simulate mean/concept shifts; train a domain classifier; backtest seasonality; determine whether drift harms accuracy.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Drift laboratory	Compare detectors on shifts	NumPy, sklearn	Statistical depth
Sensor watchdog	Streaming change alerts	River; UCI sensors	Online ML
Text drift monitor	Embedding/domain shifts	Hugging Face; dated news	NLP operations

19. Quick Revision

Compare aligned populations and confirm impact. Formulas: PSI, KS, JS. Metrics: effect, false alerts, delay, degradation relation. Trap: drift $\neq$ failure. One-liner: "Drift is a triage signal, not an automatic retraining command."

20. Final Cheat Sheet

Input/output: two windows to scores/alerts. Steps: validate, align, compare, correct, investigate, act. Knobs: reference/window, bins/kernel, alpha/effect threshold. Pros: early warning; cons: false positives and weak causal meaning.

Logging

1. Overview

Logging emits discrete structured events explaining what an ML system did and why it failed. Useful logs connect request, model, feature, deployment, and outcome through correlation IDs while protecting secrets and personal data.

2. Intuition

Metrics say "2% failed"; logs are selected witness statements explaining which failures, versions, categories, and dependencies were involved.

3. Prerequisites

Errors, JSON, log levels, IDs, distributed tracing, retention/access control, PII security, and centralized search.

4. Core Concepts

Concept	Meaning / importance	Interview angle
Structured log	Typed JSON fields	Reliable search/aggregation
Level	DEBUG/INFO/WARNING/ERROR	Noise and leakage control
Correlation ID	Connects services	Never a metric label
Context	version, route, region, latency	Incident slicing
Redaction	Remove/hash sensitive values	Privacy boundary
Sampling/retention	Control volume	Preserve errors/audits

5. Algorithm / Working Process

Define event schema/threat model; create/propagate request ID; log lifecycle boundaries; attach immutable version and safe dimensions; sanitize before serialization; ship asynchronously; restrict access/retention; correlate traces/metrics; review failures and cost.

6. Mathematical Foundation

```
bytes/day = requests/second * events/request * avg_event_bytes * 86400
```

If normal requests are sampled with probability s , unbiased count estimates weight sampled events by $1/s$. Errors commonly use $s=1$.

7. Practical Implementation

```
import json, logging, uuid

logging.basicConfig(level=logging.INFO, format="%(message)s")
log = logging.getLogger("inference")

def emit(event, **fields):
    forbidden = {"password", "token", "raw_text"}
    safe = {k: ("[REDACTED]" if k in forbidden else v)
            for k, v in fields.items()}
    log.info(json.dumps({"event": event, **safe}, default=str))

emit("prediction_completed", request_id=uuid.uuid4().hex,
     model_version="12", latency_ms=18.2)
```

8. Code Explanation

JSON creates queryable fields. Redaction is illustrative; production should allow-list fields and add platform filters. Configure handlers once, not per request.

9. Training / Evaluation

Training logs need run/step, loss summaries, checkpoint/data versions, and failures—not every sample. Serving needs IDs, version, timings, status, and fallback. Test redaction, schema parsing, context propagation, and log-sink outage behavior.

10. Complexity and Cost

Cost is $O(\text{events} \times \text{size})$; synchronous network logging hurts latency. Buffer/batch, rotate, sample success, preserve errors/audits, and cap traces. Search/egress can exceed inference cost.

11. Common Use Cases

Incident diagnosis, version audit, fallback analysis, batch partition failure, feature timeout, LLM safety review, and appeal tracing.

12. Common Mistakes

Concatenated strings, raw payload/secrets, no request/model ID, logs as metrics, high-cardinality floods, swallowed exceptions, duplicate handlers, and continuing after corruption.

13. Edge Cases / Limitations

Logs can be lost, reordered, or become a security liability. Sampling can omit a rare chain. Exact user audit may require an append-only ledger rather than debug logs.

14. Variations

Logs describe events, metrics aggregate numbers, traces show distributed timing, and audit logs record actions immutably. OpenTelemetry correlates them.

15. Related Topics

Monitoring aggregates signals; distributed serving needs trace context; reproducibility uses run logs but depends on manifests; privacy defines fields/retention.

16. Interview Questions

1. **Why structured?** Typed fields support consistent search.
2. **Include?** Time, IDs, version, route, status, latency, safe diagnostics.
3. **Exclude?** Secrets, PII, unrestricted prompts/documents.
4. **Log versus metric?** Event detail versus aggregation.
5. **Why correlation ID?** Connect one operation across services.
6. **Avoid latency?** Async buffering/batching.
7. **Control cost?** Levels, sampling, schemas, retention.

8. **Why no ID metric label?** Unbounded cardinality.
9. **How test?** Schema, redaction, context, sink outage.
10. **Audit versus debug?** Durable controlled evidence versus operational detail.

17. Practice Tasks

Implement JSON logs; propagate IDs; add allow-list redaction; calculate daily volume; trace a timeout across services.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Inference audit trail	Safe searchable events	FastAPI, OpenTelemetry	Observability/security
Pipeline explorer	Correlate run/partition errors	Python, Loki	Data operations
LLM privacy logger	Metadata/redaction tests	FastAPI, Presidio	Responsible AI

19. Quick Revision

Small, structured, correlated, privacy-safe events. Formula: daily volume. Trap: high cardinality/PII. One-liner: "Metrics tell me where; correlated logs tell me what."

20. Final Cheat Sheet

Input/output: runtime event to JSON. Knobs: level, sampling, retention, schema. Pros: diagnosis/audit; cons: cost, latency, privacy. Prefer allow-listed fields and safe degradation.

Reproducibility

1. Overview

Reproducibility means another run can recreate sufficiently equivalent data, pipeline, environment, artifact, and evaluation. Exact reproducibility demands identical results; statistical reproducibility demands conclusions within stochastic variation.

2. Intuition

A seed is one ingredient, not the recipe. Reproduction needs the kitchen, ingredient batches, equipment versions, steps, and measurement procedure.

3. Prerequisites

RNGs, Git, environment locks, versioning, deterministic algorithms, floating point, pipelines, and tracking.

4. Core Concepts

Concept	Meaning / importance	Interview angle
Seed control	Python/NumPy/framework/worker RNG	Necessary, insufficient
Environment lock	Packages, OS/CUDA/driver	Runtime differences
Determinism	Stable operation/order	Performance tradeoff
Data/split identity	Immutable bytes/membership	Common missing dependency
Statistical repeat	Distribution across seeds	Research evidence
Manifest	Links all inputs/outputs	Rebuild contract

5. Algorithm / Working Process

Pin code and clean diff; version data/labels/split/features/evaluator; lock dependencies/container/hardware; seed before construction; choose deterministic operations; persist config/commands; hash outputs; rerun in a clean environment; compare hashes or tolerances/intervals.

6. Mathematical Foundation

```

mean(M) = sum_i M_i / k
s2 = sum_i (M_i - mean)2 / (k - 1)
SE = s / sqrt(k)
(a + b) + c may not equal a + (b + c)

```

Parallel floating-point reduction order can change low bits and later optimization paths.

7. Practical Implementation

```

import os, random
import numpy as np

def seed_everything(seed=42):
    os.environ["PYTHONHASHSEED"] = str(seed)
    random.seed(seed)
    np.random.seed(seed)
    try:
        import torch
        torch.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        torch.use_deterministic_algorithms(True)
    except ImportError:
        pass

```

8. Code Explanation

This covers common RNGs and deterministic PyTorch algorithms. Run before constructing loaders/models. Some operations raise without a deterministic implementation and may be slower. Environment/data/worker seeding still matter.

9. Training / Evaluation

Persist split indices and evaluator version. Compare exact hashes/tolerances when required. For research, repeat seeds and report mean, dispersion, CI, and selection rule. Rebuild from a clean checkout/container.

10. Complexity and Cost

Deterministic kernels can reduce throughput; repeated seeds multiply compute; immutable data/checkpoints use storage. Apply exact determinism to tests/audits and statistical reproduction to noisy training.

11. Common Use Cases

Papers, regulated models, incident reconstruction, handoff, CI golden tests, benchmarks, and LLM fine-tuning.

12. Common Mistakes

Only NumPy seed, recreated random splits, unpinned transitive packages, mutable queries, notebook state, missing preprocessing, best-seed cherry-picking, and assuming cross-hardware bitwise identity.

13. Edge Cases / Limitations

External APIs change; GPU collectives/atomics may be nondeterministic; upgrades change numerics; legal deletion removes data; live features may be unreplayable.

14. Variations

Computational reproducibility reruns code/data; statistical reproducibility repeats conclusions; replicability uses independent methods/data. Hermetic builds maximize isolation.

15. Related Topics

Tracking records manifests, DVC data, Docker runtime, CI clean rebuilds, and model versioning hashes outputs. Reproducible does not mean correct.

16. Interview Questions

1. **Seed enough?** No—data, split, code, environment, algorithms, hardware matter.
2. **Exact versus statistical?** Identical output versus consistent conclusion.
3. **GPU nondeterminism?** Scheduling, atomics, reduction order.
4. **Lockfile?** Resolved dependency versions/hashes.
5. **Rebuild procedure?** Clean environment from manifest; compare defined outputs.
6. **Save split indices?** Seed/library/order may recreate differently.
7. **Report seeds?** Mean, SD/CI, count, selection rule.
8. **Container enough?** No; data/config/external services/drivers remain.
9. **Why command/config?** Defaults and invocation alter behavior.
10. **Tradeoff?** Determinism/repeats cost performance and compute.

17. Practice Tasks

Reproduce sklearn output; expose unseeded failure; containerize; compare five seeds; build a manifest verifier.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Repro benchmark	One-command exact rebuild	DVC, Docker, MLflow; Iris	Discipline
Seed stability	DL confidence intervals	PyTorch; Fashion-MNIST	Research rigor
Incident replay	Historical prediction rebuild	Event log, registry	Forensics

19. Quick Revision

Pin every material input and define equivalence. Formula: mean/SE across seeds. Trap: seed-only claims. One-liner: "Reproducibility is a manifest plus a clean rerun and explicit equality criterion."

20. Final Cheat Sheet

Inputs: code/data/config/environment/seed/hardware. Output: equivalent artifact/metrics. Knobs: deterministic algorithms, tolerance, repeats. Pros: trust/debugging; cons: compute/storage/slower kernels.

Docker

1. Overview

Docker builds a layered image containing code, runtime, libraries, and defaults, then runs it as an isolated process. In ML it packages APIs, trainers, batch jobs, and model servers.

2. Intuition

An image is a sealed shipping template; a container is one running shipment. It packages dependencies, not a full VM, and shares the host kernel.

3. Prerequisites

Linux processes/filesystems, images, ports, env vars, packages, registries, volumes, and security.

4. Core Concepts

Concept	Meaning / importance	Interview angle
Layer/cache	Filesystem delta	Stable installs before code
Base image	Runtime start	Pin digest/minimize surface
Multi-stage	Separate build/runtime	Smaller secure image

Concept	Meaning / importance	Interview angle
ENTRYPOINT/CMD	Executable/default args	Exec form handles signals
Volume	Runtime external data	Image stays immutable
User	Non-root identity	Limit compromise

5. Algorithm / Working Process

Choose trusted pinned base; copy dependency manifests; install locked dependencies; copy needed source; create non-root user; define health and exec-form command; build/tag/digest; scan/sign; run with injected secrets, limits, and port; test startup/signals/artifact access.

6. Mathematical Foundation

```
memory_total approximately runtime_base + workers * (model + overhead)
```

Copy-on-write may share CPU pages, but GPU memory is commonly duplicated. SHA-256 image digest gives immutable identity.

7. Practical Implementation

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py model.joblib ./
RUN useradd --create-home appuser
USER appuser
EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

8. Code Explanation

Dependencies precede source for cache reuse. Slim/no-cache reduce size. Non-root and exec CMD improve security/signals. Production pins base digest/package hashes and may verify-download large models at startup.

9. Training / Evaluation

Build cleanly in CI. Run tests inside the image, vulnerability/license scans, size/health/non-root/read-only checks, and graceful SIGTERM. Benchmark numerical parity and latency.

10. Complexity and Cost

CUDA/model layers slow build/push/pull/cold start. Cache and multi-stage builds help. Embedded weights improve atomicity; external weights improve reuse but need verified readiness.

11. Common Use Cases

APIs, MLflow/Kubeflow jobs, GPU inference, reproducible training, local stacks, and CI.

12. Common Mistakes

latest tags, copying repo/secrets, root, startup installs, mutable versions, runtime state in image, and ignored SIGTERM.

13. Edge Cases / Limitations

GPU driver stays external; host kernel is shared; architectures differ; huge models challenge distribution; secrets remain recoverable from old layers.

14. Variations

Multi-stage, distroless, Podman/buildah, GPU CUDA images, and Compose. Distroless is smaller but harder to debug.

15. Related Topics

CI builds/scans, registries store images, Kubernetes runs containers, model registries store artifacts, reproducibility records digests.

16. Interview Questions

1. **Image/container?** Template/running process.
2. **Container/VM?** Shared kernel/guest OS.
3. **Layer order?** Cache reuse.
4. **Non-root?** Limits compromise.
5. **Tag/digest?** Mutable name/immutable identity.
6. **COPY/volume?** Build content/runtime mount.
7. **Secrets?** Runtime secret manager, never image.
8. **Exec CMD?** Correct signals/arguments.
9. **Where model?** Image if small/atomic; verified artifact store if large/shared.
10. **Docker not solve?** Data lineage, orchestration, validity, hardware.

17. Practice Tasks

Containerize API; inspect layers; run non-root/read-only; test SIGTERM; compare model-loading approaches.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Hardened image	Small scanned API	Docker, FastAPI; Iris	Hygiene

Project	What it does	Stack/dataset	Resume value
GPU matrix	Runtime compatibility tests	Docker, PyTorch	Platform skill
Local ML stack	API/tracker/object store	Compose, MLflow, MinIO	Integration

19. Quick Revision

Immutable runtime packaging. Memory often grows per worker; digest identifies content. Traps: latest, root, secrets, huge context. One-liner: “Deploy by digest, run non-root, lock dependencies, test health/signals.”

20. Final Cheat Sheet

Input/output: Dockerfile to image to container. Knobs: base, layers, user, command, limits. Pros: portable/repeatable; cons: size, security, driver concerns.

CI/CD for ML

1. Overview

CI validates code, data contracts, pipelines, models, and packaging. CD promotes approved artifacts. ML adds expensive training, statistical gates, lineage, and nondeterminism to software delivery.

2. Intuition

CI is inspection; CD is the controlled conveyor. Training produces a candidate—it does not grant production trust.

3. Prerequisites

Git, tests, builds, containers, registries, environments, secrets, rollouts, metrics, reproducibility.

4. Core Concepts

Concept	Meaning / importance	Interview angle
CI trigger	PR/code/config change	Fast feedback
CT trigger	Schedule/data/drift/manual	Training differs from deployment
Quality gate	Contract/metric/slice/latency/security	Statistical tolerance
Promotion	Same immutable bytes	Never rebuild production
GitOps	Desired state in Git	Audit/rollback
Progressive delivery	Shadow/canary/A-B	Limit blast radius

5. Algorithm / Working Process

PR: lint/type/unit/contract tests, small training smoke test, build/scan. Training trigger: version data, train/evaluate candidate, compare champion, register. Approval: policy/manual based on risk. Promote same digest to staging, integrate/load/shadow-test, canary production, halt/rollback on breach, record deployment.

6. Mathematical Foundation

```
promote if lower_confidence_bound(metric_new - metric_old) > -epsilon
and every hard constraint passes
expected_canary_samples = traffic_fraction * request_rate * duration
```

Duration must cover statistical power and business cycles.

7. Practical Implementation

```
name: ml-ci
on: [pull_request]
jobs:
  test-and-build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.12"}
      - run: pip install -r requirements.txt
      - run: pytest -q
      - run: python train.py --smoke-test
      - run: docker build -t risk-api:commit-sha .
```

8. Code Explanation

The PR path is bounded: tests and smoke training catch integration issues without full cost. Production captures image digest, scans/signs it, and promotes instead of rebuilding. Secrets remain in the CI store.

9. Training / Evaluation

Separate fast deterministic PR checks from full evaluation. Gate baseline/slices/calibration/data quality/golden inference/reproducibility/security/latency/memory/cost. Test pipeline code on representative fixtures.

10. Complexity and Cost

Full training per commit wastes compute. Use change triggers, smoke data, scheduled candidates, parallel checks, cache, and budgets. Track pipeline duration, queue time, failure, and flaky rate.

11. Common Use Cases

Forecast retraining, fraud models, API releases, feature changes, LLM prompt/index evaluation, regulated promotion.

12. Common Mistakes

Deploying on score alone, rebuilds, latest tags, test tuning, YAML secrets, no rollback, flaky gates, full training every PR, approval without evidence.

13. Edge Cases / Limitations

Delayed labels, scarce GPUs, data-only changes, nondeterminism, and schema migrations complicate delivery. Use backward-compatible expand-contract changes.

14. Variations

Continuous delivery stops before manual production approval; continuous deployment releases passing changes; continuous training builds candidates. GitHub/GitLab/Jenkins and Argo/Kubeflow are implementations.

15. Related Topics

Docker supplies artifacts; registries promote; DVC/MLflow give lineage; Kubernetes rolls out; A/B/shadow reduce risk; monitoring can trigger rollback.

16. Interview Questions

1. **ML CI/CD difference?** Adds data/model behavior and learned artifacts.
2. **CT versus CD?** Candidate training versus release.
3. **Promote same artifact?** Rebuild can invalidate evidence.
4. **PR strategy?** Fast tests plus tiny end-to-end smoke.
5. **Promotion blockers?** Quality, slices, latency, security, cost, compatibility.
6. **Nondeterminism?** Tolerances, seeds, confidence bounds.
7. **Rollback?** Retained compatible artifact and traffic reversal.
8. **Retrain triggers?** Schedule, labels, verified drift/performance, manual.
9. **Secrets?** Short-lived CI identity and secret store.
10. **Why canary?** Limit exposure while measuring production.

17. Practice Tasks

Create PR smoke pipeline; add metric gate; publish by digest; simulate rollback; optimize expensive workflow.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Model release	Train/register/canary/rollback	Actions, MLflow, FastAPI	End-to-end
Data-contract CI	Validate feature changes	DVC, Pandera; Adult	Reliability
RAG gate	Block prompt/index regressions	pytest, MLflow; QA	GenAI quality

19. Quick Revision

Fast CI, governed training, same-artifact promotion, progressive delivery. Formula: canary samples = fraction × rate × duration. Trap: trained ≠ deployable. One-liner: "Training emits a candidate; evidence and policy promote it."

20. Final Cheat Sheet

Inputs: code/data/config/model changes. Outputs: tested registered artifact/deployment. Knobs: triggers, gates, approvals, canary. Pros: safe repeatability; cons: compute/pipeline complexity.

MLflow

1. Overview

MLflow is an open-source MLOps platform whose major capabilities are experiment tracking, model packaging/signatures, artifact storage, and a model registry. It is framework-agnostic and can run locally or with a shared tracking server, metadata database, and object store.

2. Intuition

MLflow is a searchable lab notebook connected to a release catalog: runs store evidence; registered models store selected deployable versions.

3. Prerequisites

Python, training runs, parameters/metrics/artifacts, URLs, databases/object storage, signatures, registries, and access control.

4. Core Concepts

Concept	Meaning	Interview angle
Tracking URI/backend	Run metadata service/database	SQLite versus shared DB
Artifact store	Models/plots/files	Credentials and immutable storage
Experiment/run	Group/execution	Nested runs for tuning
Autologging	Framework hooks	Convenient but inspect output
MLflow Model	Flavor plus metadata	pyfunc portable interface
Registry alias	Champion/candidate pointer	Resolve then pin version

5. Algorithm / Working Process

Configure tracking URI/experiment; start a run; log code/data tags and parameters; train; log metrics by step and artifacts; infer/log signature and input example; end status; compare compatible runs; register chosen artifact; run gates; assign alias; serve/export the pinned version.

6. Mathematical Foundation

MLflow does not change training mathematics. It records θ and observed $M(\theta)$. Selection is $r^* = \arg\max$ validation metric, with test evaluation after selection. Logging mean/SD across seeds avoids mistaking noise for improvement.

7. Practical Implementation

```
import mlflow
import mlflow.sklearn
from mlflow.models import infer_signature

with mlflow.start_run():
    model.fit(X_train, y_train)
    predictions = model.predict(X_valid)
    mlflow.log_params({"max_depth": model.max_depth})
    mlflow.log_metric("validation_accuracy",
                     float((predictions == y_valid).mean()))
    signature = infer_signature(X_train, model.predict(X_train))
    mlflow.sklearn.log_model(
        model, "model", signature=signature, input_example=X_train[:2]
    )
```

8. Code Explanation

The context manager marks completion/failure. Params are immutable run inputs; metrics are measured outputs. The signature and example let downstream tools validate serving shape/types. Add data digest, Git commit, split ID, requirements, and tags.

9. Training / Evaluation

Use one parent run per study and nested child runs for candidates. Log split/evaluator versions, learning curves, slices, calibration, predictions or hashes, runtime, hardware, and cost. Compare only like-for-like runs and register a complete inference pipeline.

10. Complexity and Cost

Frequent metric writes burden the tracking server; batch/sample them. Large checkpoints belong in object storage with lifecycle policies. Shared servers require DB backups, artifact durability, auth, TLS, and retention.

11. Common Use Cases

Hyperparameter comparison, model registry, framework-independent packaging, team dashboards, CI promotion, and LLM/RAG evaluation tracking.

12. Common Mistakes

Local file store in team production, missing data/split tags, secrets in artifacts, registering weights without preprocessing, autologging blindly, and moving an alias without audit/gates.

13. Edge Cases / Limitations

MLflow is not a full workflow orchestrator or feature store. Concurrent/local SQLite has limits. Custom models need pyfunc wrappers and dependency care. Registry semantics/configuration vary by deployment/version.

14. Variations

Local MLflow suits practice; remote server separates backend/artifacts; managed MLflow adds operations; pyfunc provides generic predict; framework flavors preserve native loading. Core concepts are placement-important.

15. Related Topics

Tracking, model registry, versioning, Docker, CI/CD, and serving. DVC focuses data/pipeline versioning; Kubeflow orchestrates Kubernetes workflows.

16. Interview Questions

1. **Tracking backend/artifact store?** Metadata database versus large-file storage.
2. **Experiment/run?** Study grouping versus one execution.
3. **Param/metric/artifact?** Input scalar, observed scalar series, file.
4. **Flavor?** Framework-specific loading metadata/API.
5. **pyfunc?** Common Python predict interface.
6. **Why signature?** Input/output contract validation.
7. **Registry alias?** Mutable semantic pointer to immutable version.
8. **Nested runs?** Organize tuning trials.
9. **Can MLflow schedule training?** Not primarily; use an orchestrator.
10. **Production architecture?** Server + durable DB + object store + auth/TLS.

17. Practice Tasks

Track three classifiers; add signatures; run nested tuning; register winner; promote/rollback an alias.

18. Project Ideas

| Project | What/stack/data | Resume value | |---|---| | Churn tracker | MLflow + sklearn; Telco | Experiment discipline | | Registry delivery | MLflow + CI + FastAPI; Adult | Governed release | | RAG experiments | MLflow + retrieval evaluator; BEIR | LLMOps |

19. Quick Revision

MLflow links runs to artifacts and governed versions. Key identifiers: experiment, run ID, artifact URI, registered model/version/alias. Trap: tracker is not orchestrator. One-liner: "Metadata lives in the backend; large artifacts live in the artifact store."

20. Final Cheat Sheet

Input/output: training execution to searchable run/model version. Steps: start, log, compare, register, alias. Knobs: URI, experiment, tags, signature. Pros: framework-neutral lineage; cons: shared-service operations and incomplete orchestration.

DVC

1. Overview

DVC (Data Version Control) versions large data/model artifacts through small Git-tracked metadata files, a content-addressed cache, and optional remote storage. It also defines reproducible pipeline stages and metrics/plots.

2. Intuition

Git stores lightweight claim tickets; DVC stores heavy luggage in a remote warehouse keyed by content. Checking out a commit tells DVC which exact luggage to fetch.

3. Prerequisites

Git, hashes, caches, object storage, command-line workflows, data pipelines, YAML, and reproducibility.

4. Core Concepts

Concept	Meaning	Interview angle
.dvc file	Pointer/metadata for tracked output	Commit it, not large bytes
Cache	Content-addressed local objects	Deduplication
Remote	Shared durable object storage	Auth/config
dvc.yaml	Stage commands/deps/outs/params	Rebuild DAG
dvc.lock	Resolved hashes and commands	Reproduction record
repro/push/pull	Rebuild/share/fetch	Git and DVC workflow

5. Algorithm / Working Process

Initialize in Git; add data with DVC; configure a remote; commit pointer/config; push objects. Define stages with commands, dependencies, parameters, outputs, metrics; run `dvc repro`, which executes changed stages and updates lock; commit metadata; reproduce an old revision by Git checkout plus `dvc pull/repro`.

6. Mathematical Foundation

Content addressing maps bytes B to $H(B)$; unchanged objects deduplicate. Pipeline invalidation follows a DAG: rerun stage s if its command, parameters, dependency hash, or output declaration changes, then invalidate descendants.

7. Practical Implementation

```
stages:
  prepare:
    cmd: python prepare.py
```

```
deps: [data/raw.csv, prepare.py]
outs: [data/processed.csv]
train:
  cmd: python train.py
  deps: [data/processed.csv, train.py]
  params: [train.max_depth]
  outs: [models/model.joblib]
  metrics: [metrics.json]
```

```
dvc repro
dvc push
git add dvc.yaml dvc.lock metrics.json
```

8. Code Explanation

The DAG reruns prepare when raw data/code changes and train when its data/code/parameter changes. dvc.lock pins resolved identities. Outputs are cached/remotely stored; small metadata is committed in Git.

9. Training / Evaluation

Version raw/processed data, split definitions, params, model, and metrics. Test a clean clone with dvc pull and dvc repro. Use temporal splits and avoid committing sensitive content through accidental Git tracking.

10. Complexity and Cost

Hashing is $O(\text{bytes})$; initial push/pull is network-heavy. Cache deduplicates identical content but many changed large files still cost storage. Pipelines save compute by rerunning only invalidated stages.

11. Common Use Cases

Dataset sharing, reproducible papers, file-based feature pipelines, model artifacts, experiments tied to Git branches, and CI data pulls.

12. Common Mistakes

Committing data to Git too, forgetting dvc push, mutable/underprotected remote, ignoring dvc.lock, declaring incomplete dependencies, and storing credentials in tracked config.

13. Edge Cases / Limitations

Rapid streams and warehouse tables fit snapshot-native tools better. Millions of tiny files add overhead. Team cache permissions and remote garbage collection require care. DVC tracks bytes, not semantic correctness.

14. Variations

DVC versus Git LFS: DVC adds pipelines/experiments/remote flexibility; lakehouse time travel versions tables; Pachyderm/lakeFS provide data-platform lineage. DVC is ideal for placement demos.

15. Related Topics

Data versioning is the concept; DVC implements file workflows. MLflow tracks runs; both can coexist. CI invokes `dvc repro`; Docker pins runtime.

16. Interview Questions

1. **Why not Git?** Large binary history performs poorly.
2. **What is .dvc?** Git-tracked pointer metadata.
3. **Cache/remote?** Local content store/shared durable store.
4. **dvc.yaml/lock?** Declared DAG/resolved state.
5. **What triggers rerun?** Changed command, parameter, dependency, or declaration.
6. **How reproduce commit?** Checkout, `dvc pull`, `reproduce/execute`.
7. **DVC versus MLflow?** Data/pipeline versioning versus run/registry tracking.
8. **Credentials?** Untracked local config or workload identity.
9. **Forgot push?** Metadata exists but teammates cannot fetch objects.
10. **Does hash validate quality?** No, only content identity.

17. Practice Tasks

Track data; configure local remote; create two-stage DAG; reproduce old commit; simulate missing push.

18. Project Ideas

| Project | What/stack/data | Resume value | |---|---| | Repro classifier | DVC/sklearn; Adult | Data lineage | | Image preprocessing DAG | DVC/OpenCV; CIFAR | Pipeline caching | | RAG corpus versioning | DVC/FAISS; documents | Corpus/index lineage |

19. Quick Revision

DVC keeps lightweight metadata in Git and content in cache/remote. Key commands: `add`, `repro`, `push`, `pull`. Trap: lock committed but objects not pushed. One-liner: "Git selects the metadata revision; DVC materializes its large content."

20. Final Cheat Sheet

Input/output: files/commands to content IDs and reproducible DAG. Knobs: remote, cache, dependencies, params, outputs. Pros: simple Git-aligned lineage; cons: storage/network and weaker fit for streams/tables.

FastAPI

1. Overview

FastAPI is a Python ASGI framework for typed APIs. Type hints and Pydantic provide validation and OpenAPI documentation; ASGI supports concurrent I/O. It is popular for ML endpoints but does not itself provide model registry, autoscaling, GPU scheduling, or monitoring.

2. Intuition

Function annotations become a guarded, documented doorway: invalid requests stop at the entrance before expensive inference.

3. Prerequisites

Python typing, HTTP/REST, Pydantic, sync versus async, model loading, process workers, and testing.

4. Core Concepts

Concept	Meaning	Interview angle
Pydantic model	Typed validated schema	Trust boundary
ASGI	Async server interface	I/O concurrency
Lifespan	Startup/shutdown resource management	Load model once
Dependency injection	Shared auth/clients	Testability
OpenAPI	Generated contract/docs	Client/schema governance
TestClient	In-process endpoint tests	Contract tests

5. Algorithm / Working Process

Define schemas and bounds; load/verify/warm model in lifespan; expose liveness/readiness; validate/authenticate request; call sync inference or offload appropriately; map domain errors; return versioned schema; add middleware for IDs/limits/telemetry; run behind a production ASGI server and proxy.

6. Mathematical Foundation

Little's Law gives $\text{concurrency} = \lambda W$. Async improves utilization during I/O wait, not CPU-bound model execution. With k worker processes, memory can approach k times model memory.

7. Practical Implementation

```

from contextlib import asynccontextmanager
from fastapi import FastAPI, Request
from pydantic import BaseModel, Field

state = {}

@asynccontextmanager
async def lifespan(app):
    state["model"] = load_verified_model()
    yield
    state.clear()

app = FastAPI(lifespan=lifespan)

class Features(BaseModel):
    values: list[float] = Field(min_length=4, max_length=4)

```

```
@app.post("/v1/predict")
def predict(x: Features, request: Request):
    y = state["model"].predict([x.values])[0]
    return {"prediction": float(y), "request_id": request.headers.get("x-request-id")}
```

8. Code Explanation

Lifespan loads once before readiness and cleans up. Bounded list length prevents shape errors. A regular def endpoint is suitable for blocking CPU work because FastAPI runs it in a threadpool; measure and use process/model-server architecture for heavy compute.

9. Training / Evaluation

Keep training outside the API. Test Pydantic boundary values, golden predictions, readiness before load, malformed/oversized requests, concurrent load, timeouts, error mapping, and shutdown.

10. Complexity and Cost

Validation is linear in payload size; inference dominates. Threads share model memory but Python CPU execution may face the GIL; processes parallelize but duplicate memory. GPU models often need one controlled model-serving worker with batching.

11. Common Use Cases

Prediction APIs, async job submission, feature/registry gateways, internal ML services, and prototypes.

12. Common Mistakes

Loading per request, async def with blocking inference, unbounded lists/files, global mutable per-request data, training in endpoint, dev reload in production, and too many memory-duplicating workers.

13. Edge Cases / Limitations

Large uploads, streaming tokens, client cancellation, multiprocessing GPU context, and long tasks need specialized handling. FastAPI is an app framework, not an orchestration platform.

14. Variations

Sync endpoints suit blocking calls; async suits awaited I/O; BackgroundTasks suits tiny post-response work, not durable jobs; queues suit long work; SSE/WebSocket streams tokens.

15. Related Topics

REST defines the contract; Uvicorn serves ASGI; Docker packages it; Kubernetes operates it; MLflow provides artifacts; OpenTelemetry observes it.

16. Interview Questions

1. **Why FastAPI for ML?** Validation/docs and simple Python integration.
2. **ASGI?** Async server-app protocol.
3. **When async?** Awaitable I/O, not merely because it looks faster.
4. **Load model where?** Lifespan/startup once per process.
5. **How validate shape?** Pydantic bounds plus semantic checks.
6. **Multiple workers issue?** Model memory duplication, especially GPU.
7. **Long job?** Queue and 202 job ID.
8. **Health routes?** Liveness and model/dependency readiness.
9. **How test?** TestClient/contracts/golden/load/failure tests.
10. **Does FastAPI autoscale?** No; deployment platform does.

17. Practice Tasks

Build endpoint; add lifespan; validate batch cap; test 422/503; benchmark sync/async misuse.

18. Project Ideas

| Project | What/stack/data | Resume value | |---|---| | Risk API | FastAPI/sklearn; credit data | Contracts | | Image API | FastAPI/PyTorch; CIFAR | Upload/inference | | RAG streaming API | FastAPI/SSE/vector DB | GenAI endpoint |

19. Quick Revision

Typed ASGI APIs with Pydantic/OpenAPI. Math: $\text{concurrency} = \text{lambda} \times \text{latency}$. Trap: blocking work in async and per-worker model duplication. One-liner: "FastAPI validates transport; the predictor owns ML behavior."

20. Final Cheat Sheet

Input/output: typed HTTP request/response. Knobs: schema, lifespan, worker count, timeout, batch limit. Pros: productive/docs; cons: not a full serving platform.

Kubernetes Basics

1. Overview

Kubernetes reconciles declared workload state across a cluster. It schedules containers, restarts failures, exposes services, performs rolling updates, manages configuration/secrets references, and autoscaling. For ML it operates APIs, model servers, batch jobs, and pipeline components.

2. Intuition

You declare "three healthy copies"; controllers continuously compare desired and actual state and repair the difference.

3. Prerequisites

Containers, YAML, networking, CPU/memory/GPU resources, health checks, rolling releases, and distributed systems.

4. Core Concepts

Object	Purpose	Interview angle
Pod	Smallest scheduled unit	Ephemeral, often one app container
Deployment	Replica/rolling controller	Stateless serving
Service	Stable virtual endpoint	Selects pod labels
ConfigMap/Secret	External configuration	Secret needs encryption/RBAC
Job/CronJob	Finite/scheduled task	Batch inference/training
requests/limits	Scheduler/QoS constraints	OOMKill/throttling
probes	startup/liveness/readiness	Different failure meanings
HPA	Replica autoscaling	Custom queue/GPU signals

5. Algorithm / Working Process

Build/push image by digest; define Deployment with requests/limits/security; mount config/secret; configure startup/readiness/liveness; expose Service; apply desired state; scheduler assigns nodes; controllers replace failures and roll versions; HPA changes replicas; monitor and roll back.

6. Mathematical Foundation

Required replicas can be approximated by:

```
replicas >= ceil(peak_QPS * p99_service_time / safe_concurrency_per_pod)
```

Requests drive scheduling; limits constrain use. Availability requires enough ready replicas across independent failure domains.

7. Practical Implementation

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: risk-api}
spec:
  replicas: 3
  selector: {matchLabels: {app: risk-api}}
  template:
    metadata: {labels: {app: risk-api}}
    spec:
      containers:
        - name: api
          image: registry/risk-api@sha256:immutable-digest
          ports: [{containerPort: 8000}]
          resources:
```

```
requests: {cpu: "500m", memory: "1Gi"}
limits: {memory: "2Gi"}
readinessProbe:
  httpGet: {path: /health/ready, port: 8000}
```

8. Code Explanation

Deployment owns three replicas. Labels connect selector and pods. Digest pins bytes. Requests reserve scheduling capacity; memory limit bounds use. Readiness prevents traffic until model load/warm-up completes.

9. Training / Evaluation

Test manifests with schema/policy checks, deploy to a temporary namespace, validate probes, resource limits, load, disruption, rolling update, rollback, and node/GPU placement. Benchmark requests from client perspective.

10. Complexity and Cost

Each replica consumes requested capacity; headroom improves reliability but costs money. Huge images/models slow scheduling. GPU fragmentation and one-model-per-pod waste capacity; batching/multi-model servers can improve use with isolation tradeoffs.

11. Common Use Cases

Autoscaled APIs, GPU serving, scheduled scoring, distributed training jobs, Kubeflow pipelines, and multi-tenant ML platforms.

12. Common Mistakes

No resource requests, liveness during slow startup, readiness that ignores model load, mutable image tags, secrets in manifests, local pod storage for artifacts, CPU-based GPU autoscaling, and no disruption budget.

13. Edge Cases / Limitations

Stateful model caches, long downloads, GPU drivers/node pools, scale-to-zero cold starts, and multi-region consistency add complexity. Kubernetes does not validate model quality.

14. Variations

Deployment for stateless long-running work, StatefulSet for stable identity/storage, Job/CronJob for finite work, DaemonSet per node. KServe/Ray Serve add model-serving abstractions.

15. Related Topics

Docker packages; Kubernetes schedules; Kubeflow builds ML workflows; distributed serving runs above it; CI/CD updates desired state; monitoring observes it.

16. Interview Questions

1. **Pod/Deployment?** Runtime unit/controller maintaining replicas.
2. **Service?** Stable discovery/load-balancing endpoint.
3. **Readiness/liveness?** Receive traffic/restart process.
4. **Startup probe?** Protect slow initialization from liveness.
5. **Request/limit?** Scheduling guarantee/usage ceiling.
6. **OOMKilled?** Container exceeded memory limit.
7. **Why digest?** Exact auditable image.
8. **How autoscale GPU serving?** Queue/in-flight/batch/GPU custom metrics.
9. **Job versus Deployment?** Finite completion versus continuous service.
10. **Rollback?** Restore prior ReplicaSet/image/config and verify readiness/SLO.

17. Practice Tasks

Deploy local API; break readiness; set low memory and inspect OOM; roll version; configure queue-based HPA design.

18. Project Ideas

| Project | What/stack/data | Resume value | |---|---| | Autoscaled inference | K8s/FastAPI/Prometheus; synthetic |
| Orchestration | | GPU model service | K8s/PyTorch; image data | Accelerator ops | | Batch CronJob | K8s/object store; churn | Batch operations |

19. Quick Revision

Declarative reconciliation of container workloads. Formula: replicas from peak load $\times$ service time / safe concurrency. Trap: wrong probes/resources. One-liner: "Readiness routes, liveness restarts, requests schedule, and limits constrain."

20. Final Cheat Sheet

Input/output: desired manifests to reconciled pods/services/jobs. Knobs: replicas, resources, probes, rollout, HPA. Pros: resilience/standardization; cons: operational complexity and cold starts.

Feature Store

1. Overview

A feature store manages reusable feature definitions and serves historically correct offline values for training plus fresh low-latency online values for inference. It addresses discovery, ownership, lineage, freshness, and training-serving consistency—not just key-value storage.

2. Intuition

It is a governed kitchen with one recipe for "customer_30d_spend," able to reconstruct what the value was yesterday and serve the latest value now.

3. Prerequisites

Feature engineering, event/entity time, SQL joins, streaming/batch ETL, online databases, leakage, schemas, and monitoring.

4. Core Concepts

Concept	Meaning	Interview angle
Entity key	customer_id, item_id	Join/lookup identity
Event timestamp	When fact became true	Point-in-time correctness
Offline store	Historical training values	Backfills/large scans
Online store	Latest low-latency values	TTL/freshness
Materialization	Copy computed values online	Consistency and lag
Feature view	Versioned schema/transform/source	Discovery/lineage

5. Algorithm / Working Process

Define entity, source, event/created time, schema, owner, TTL; compute features with versioned transform; validate; write history offline; point-in-time join for training; materialize latest values online; fetch by entity at inference; log feature-view/version; monitor freshness, nulls, skew, and latency.

6. Mathematical Foundation

For prediction time t , point-in-time value is:

```
feature(e,t) = value at max(t_event) such that
t_event <= t and t_created <= allowed_availability_time
```

Freshness lag = serve_time – newest_event_time. Online hit rate = successful complete lookups / eligible lookups.

7. Practical Implementation

```
import pandas as pd

def point_in_time_join(events, features):
    events = events.sort_values("prediction_time")
    features = features.sort_values("feature_time")
    return pd.merge_asof(
        events, features,
        left_on="prediction_time", right_on="feature_time",
        by="customer_id", direction="backward",
        allow_exact_matches=True,
    )
```

8. Code Explanation

`merge_asof` selects the latest feature no later than prediction time within each customer. Real systems also account for created/ingestion time, TTL, duplicates, timezone, and feature definition version.

9. Training / Evaluation

Validate point-in-time joins with synthetic future records, compare offline/online values on sampled entities, enforce schema/ranges/nulls, measure materialization lag, and backtest features on temporal splits. Version feature definitions and log their retrieval timestamps.

10. Complexity and Cost

Sorted historical joins are roughly $O((N+M) \log(N+M))$ including sort; online lookup aims near $O(1)$. Offline storage, backfills, stream compute, and low-latency replicas cost significantly. Use only for shared/time-sensitive features.

11. Common Use Cases

Fraud aggregates, recommender user/item features, churn histories, real-time counters, and consistent features across many models.

12. Common Mistakes

Latest-value joins for historical training, ignoring created time, duplicate definitions, no owner/TTL, online defaults differing from offline, silent stale values, and putting every column into a feature store.

13. Edge Cases / Limitations

Late/out-of-order events, corrections, hot entities, high-dimensional embeddings, cross-region consistency, and privacy deletion complicate stores. A feature store cannot eliminate semantic leakage from a bad feature.

14. Variations

Offline-only catalogs suit batch systems; online/offline stores suit real-time; stream-first stores compute continuously; feature platforms may support on-demand request features. Feast is open-source; cloud platforms offer managed stores.

15. Related Topics

Data versioning tracks datasets; feature stores manage reusable temporal features. Online/batch inference consume them; drift/freshness monitoring validates them; registries should record feature dependencies.

16. Interview Questions

1. **Why feature store?** Reuse, lineage, point-in-time training, low-latency serving.
2. **Offline/online?** Historical scans versus current key lookup.
3. **Point-in-time join?** Latest available feature not later than prediction.
4. **Event/created time?** Occurrence versus system availability.
5. **Materialization?** Publish computed features into online storage.

6. **Training-serving skew?** Different values/logic between environments.
7. **TTL?** Maximum valid age/retention for a feature.
8. **How validate parity?** Sample same entities/times across stores.
9. **Why not store all features?** Operational cost/governance without reuse need.
10. **What monitor?** freshness, availability, nulls, distributions, latency, skew.

17. Practice Tasks

Implement temporal join; inject late event; build offline/online parity test; define TTL fallback; monitor freshness.

18. Project Ideas

| Project | What/stack/data | Resume value | |---|---| | Fraud features | Feast/Redis/Parquet; fraud | Temporal rigor | | Recommender store | User/item features; MovieLens | Online features | | Streaming counters | Kafka/feature store; clicks | Real-time data |

19. Quick Revision

One governed definition for historical and online values. Formula: latest causally available event. Metrics: freshness, hit rate, parity, lookup p99. Trap: future/latest join. One-liner: "A feature store's hardest promise is point-in-time correctness, not key-value lookup."

20. Final Cheat Sheet

Input/output: sources/definitions to training datasets and online vectors. Knobs: entity, timestamps, TTL, materialization. Pros: reuse/consistency; cons: backfill and operational complexity.

A/B Testing Models

1. Overview

A/B testing randomly assigns eligible experimental units to model variants and estimates their causal effect on online outcomes. It answers whether a candidate improves user/business value, not merely whether its offline metric is higher.

2. Intuition

Serve two menus to comparable randomly chosen diners at the same time. Randomization makes the menu—not weekday, geography, or customer type—the main systematic difference.

3. Prerequisites

Probability, hypothesis tests, confidence intervals, power, randomization, business metrics, guardrails, experimentation units, and interference.

4. Core Concepts

Concept	Meaning	Interview angle
Control/treatment	Champion/candidate	Stable definitions
Unit	User/session/request/cluster	Avoid contamination
Assignment	Deterministic random bucket	Sticky allocation
Primary metric	Decision-driving outcome	Declare before test
Guardrail	Safety/latency/cost constraints	Stop harmful test
SRM	Sample-ratio mismatch	Pipeline/assignment bug
Novelty/carryover	Temporary/prior exposure effects	Duration/washout

5. Algorithm / Working Process

State hypothesis, primary metric, MDE, alpha, power, guardrails, unit, eligibility, duration, and stop rules; implement sticky random assignment; run an A/A test; launch small and verify SRM/telemetry; collect through full cycles; analyze intention-to-treat with confidence intervals; inspect predefined slices/guardrails; decide rollout, iterate, or reject; retain assignment/version evidence.

6. Mathematical Foundation

For binary conversion:

```
effect = p_treatment - p_control
SE = sqrt(p_t(1-p_t)/n_t + p_c(1-p_c)/n_c)
CI = effect ± z_(1-alpha/2) * SE
```

Approximate equal-group sample size:

```
n_per_group approximately
2 * (z_(1-alpha/2) + z_power)^2 * p_bar(1-p_bar) / MDE^2
```

Randomization estimates causal effect under consistency, no interference, and correct measurement.

7. Practical Implementation

```
import hashlib

def variant(user_id: str, experiment: str, treatment_pct=50):
    key = f"{experiment}:{user_id}".encode()
    bucket = int.from_bytes(hashlib.sha256(key).digest()[:8], "big") % 100
    return "treatment" if bucket < treatment_pct else "control"

assert variant("u1", "ranker-v2") == variant("u1", "ranker-v2")
```

8. Code Explanation

The hash gives deterministic sticky assignment without storing every bucket. Including experiment name decorrelates experiments. Production uses a central assignment service, logs eligibility/assignment/exposure, and separates assignment from actual exposure.

9. Training / Evaluation

Offline gates precede experiments. A/A validates assignment and analysis. During A/B, track primary and guardrail metrics, exposure correctness, SRM, latency, fallback, and cost. Use intention-to-treat; avoid repeatedly peeking with a fixed-horizon test unless using sequential methods.

10. Complexity and Cost

Assignment is $O(1)$. Statistical cost is traffic and opportunity loss; smaller MDE requires roughly $1/\text{MDE}^2$ samples. Long tests face seasonality and concurrent experiments. Store only needed event data under privacy policies.

11. Common Use Cases

Rankers, recommenders, fraud thresholds, notification models, pricing policies, prompts/retrievers, and UI decisions driven by models.

12. Common Mistakes

Request-level assignment for user outcomes, no exposure logging, peeking/stopping at significance, many undeclared metrics, SRM ignored, insufficient duration, changing model mid-test, and confusing correlation with randomization.

13. Edge Cases / Limitations

Network effects violate independence; rare harms lack power; strong carryover needs cluster/crossover design; regulation may forbid treatment; delayed outcomes prolong tests; bandits optimize reward but complicate unbiased inference.

14. Variations

Fixed-horizon frequentist, sequential testing, Bayesian experiments, cluster randomization, switchback tests for marketplaces, interleaving for ranking, and contextual bandits. Standard A/B is most important for placements.

15. Related Topics

Shadow deployment has no treatment effect because outputs do not drive actions. Canary checks safety but may not be powered for causal KPI. Offline evaluation screens; A/B establishes online impact.

16. Interview Questions

1. **Why randomize?** Balance confounders in expectation.
2. **Choose unit?** Unit matching treatment persistence and interference.
3. **MDE?** Smallest effect worth detecting/planning for.
4. **Power?** Probability of detecting MDE when real.
5. **SRM?** Observed group counts conflict with allocation.
6. **Assignment/exposure?** Bucketed versus actually receiving treatment.
7. **Why sticky?** Avoid inconsistent experiences/carryover.
8. **Can stop at $p < .05$?** Repeated peeking inflates false positives.
9. **Offline gain but A/B loss?** Metric mismatch, latency, feedback, population, UX.
10. **Network marketplace?** Cluster or switchback randomization.

17. Practice Tasks

Build assignment; run A/A; calculate sample size; detect SRM; analyze conversion and latency guardrail.

18. Project Ideas

| Project | What/stack/data | Resume value | |---|---| | Experiment platform | Assignment/events/analysis; synthetic | Causal systems | | Ranker interleaving | Compare search rankers; LETOR | Ranking experimentation | | RAG prompt test | Quality/cost experiment; QA | GenAI evaluation |

19. Quick Revision

Randomize eligible stable units; predeclare metric/MDE/power/guards; analyze exposure correctly. Formula: difference in means/proportions and SE. Traps: SRM, peeking, interference. One-liner: "A/B estimates causal product value; offline tests estimate predictive quality."

20. Final Cheat Sheet

Input/output: traffic + variants to causal effect/CI. Knobs: unit, allocation, MDE, alpha, power, duration. Pros: causal evidence; cons: traffic/time/risk.

Shadow Deployment

1. Overview

Shadow deployment sends a copy of real production inputs to a candidate model while the current model alone controls responses. It validates compatibility, latency, stability, resource use, and prediction disagreement without directly affecting users.

2. Intuition

A trainee pilot flies a simulator fed by the real flight instruments, while the certified pilot remains in control.

3. Prerequisites

Traffic mirroring, asynchronous systems, idempotency, privacy, production logging, versioning, load testing, and candidate/champion comparison.

4. Core Concepts

Concept	Meaning	Interview angle
Mirrored traffic	Copy after eligibility	Avoid adding user latency
No side effects	Candidate cannot write/actions	Critical safety property
Disagreement	Candidate versus champion	Slice diagnosis
Sampling	Shadow subset	Bound duplicate cost
Isolation	Separate resources/quotas	Avoid harming production
Replay gap	Mirror may miss full context	Feature consistency

5. Algorithm / Working Process

Deploy candidate isolated; asynchronously mirror sampled eligible requests with IDs/timestamps; disable all candidate side effects; apply production feature path; record candidate result/version/latency/errors; compare with champion and later labels; inspect slices and capacity; fix discrepancies; only then advance to canary/A-B.

6. Mathematical Foundation

For predictions a_i and b_i :

```
disagreement_rate = (1/n) sum_i 1[a_i != b_i]
mean_score_delta = (1/n) sum_i (p_new_i - p_old_i)
shadow_cost approximately mirror_fraction * production_inference_cost
```

Paired comparisons reduce variance because both models see the same inputs.

7. Practical Implementation

```
from concurrent.futures import ThreadPoolExecutor

pool = ThreadPoolExecutor(max_workers=4)

def serve(request, champion, challenger, log_shadow):
    response = champion.predict(request)
    # Candidate result is never on the response path.
```

```
pool.submit(lambda: log_shadow(request.id, challenger.predict(request)))  
return response
```

8. Code Explanation

Champion returns independently. Shadow execution is bounded by the executor; a real system needs a bounded queue/drop policy, timeout, isolation, redaction, and exception handling. Candidate calls must be read-only.

9. Training / Evaluation

Compare schema acceptance, errors, latency/resource distributions, score/rank/class disagreement, slice behavior, and matured-label quality. Test mirroring under overload and verify candidate failure cannot affect champion response.

10. Complexity and Cost

Mirroring fraction s roughly adds s times inference compute plus logging. Unbounded mirrors can saturate shared features/GPUs. Sample intelligently and provision isolation. Async queues introduce observation delay.

11. Common Use Cases

Runtime migrations, rewritten feature pipelines, new rankers, GPU servers, fraud models, LLM model/provider changes, and high-risk releases.

12. Common Mistakes

Candidate writes events, shadow call blocks user response, no shared request ID, shared resource starvation, comparing different feature snapshots, missing labels, and treating successful shadow as causal proof.

13. Edge Cases / Limitations

Candidate behavior may depend on downstream feedback it never receives; stochastic outputs complicate disagreement; mirroring sensitive traffic may violate policy; external API calls can incur costs/side effects; shadow cannot measure user response to candidate.

14. Variations

Live mirroring, offline traffic replay, dark launch of infrastructure only, dual-write validation, and silent prediction logging. Replay is reproducible but less production-realistic.

15. Related Topics

Canary serves a small user fraction; A/B estimates causal effect; shadow affects nobody. Monitoring and logging enable comparisons; distributed serving provides isolation.

16. Interview Questions

1. **Shadow versus A/B?** No user action versus treatment effect.

2. **Shadow versus canary?** Candidate output ignored versus used for some traffic.
3. **Why async?** Prevent candidate latency/failure from user path.
4. **Main safety rule?** No side effects.
5. **What compare?** Errors, latency, resources, outputs, slices, later labels.
6. **How correlate?** Shared request/event ID and feature timestamp.
7. **How control cost?** Sampling, quotas, bounded queues.
8. **Can it prove business lift?** No.
9. **What if candidate is slower?** Optimize/cap/batch/scale before live use.
10. **When valuable?** High-risk behavior/runtime changes with real traffic complexity.

17. Practice Tasks

Mirror asynchronously; add bounded queue; compute paired disagreement; inject candidate timeout; verify no side effects.

18. Project Ideas

| Project | What/stack/data | Resume value | |---|---| | Shadow ranker | Compare rankings on mirrored queries |
FastAPI, Redis; MovieLens | Safe rollout | | Runtime migration | sklearn versus ONNX parity | ONNX Runtime |
Numerical validation | | LLM provider shadow | Quality/latency/cost comparison | Queue, evaluators | LLMOps |

19. Quick Revision

Mirror production inputs, ignore candidate outputs, compare safely. Formula: paired disagreement and mirrored cost. Trap: hidden side effects/shared saturation. One-liner: "Shadow proves operational compatibility, not causal business value."

20. Final Cheat Sheet

Input/output: copied traffic to candidate observations. Knobs: sample, timeout, queue, isolation. Pros: realistic low user risk; cons: duplicate cost and no treatment outcome.

Kubeflow

1. Overview

Kubeflow is a Kubernetes-native ML ecosystem. Its widely used Pipelines component compiles Python-defined workflows into containerized DAG tasks with metadata, caching, parameters, artifacts, and repeatable execution. Other components may cover notebooks, distributed training, tuning, and serving depending on installation.

2. Intuition

Kubernetes runs containers; Kubeflow describes the ML assembly line connecting data preparation, training, evaluation, approval, and deployment.

3. Prerequisites

Kubernetes, containers/registries, DAGs, object storage, service accounts/RBAC, pipeline artifacts, and CI/CD.

4. Core Concepts

Concept	Meaning	Interview angle
Component	Containerized function/task	Explicit inputs/outputs
Pipeline	DAG of components	Control versus data passing
Run/experiment	Execution/group	Parameterized repeatability
Artifact	Dataset/model/metrics URI	Pass references, not huge values
Cache	Reuse identical task output	Unsafe with hidden dependencies
Metadata	Lineage/execution records	Audit/debug

5. Algorithm / Working Process

Containerize deterministic components; declare typed parameters/artifacts; compose DAG and conditions; compile pipeline spec; submit with versioned inputs/service identity/resources; scheduler executes ready tasks; store artifacts remotely and metadata centrally; cache eligible tasks; evaluate/gate candidate; register/deploy via controlled component.

6. Mathematical Foundation

A DAG $G=(V,E)$ is valid if acyclic. Critical-path duration is:

$$T_{\text{pipeline}} \geq \max \text{ over dependency paths } P \text{ of } \sum_{v \in P} T_v$$

Parallel branches reduce wall time only off the critical path. Cache key should cover code/image, command, parameters, input artifact IDs, and material environment dependencies.

7. Practical Implementation

```
from kfp import dsl

@dsl.component(base_image="python:3.12-slim")
def evaluate(metric: float, minimum: float) -> str:
    if metric < minimum:
        raise ValueError(f"quality gate failed: {metric} < {minimum}")
    return "approved"

@dsl.pipeline(name="model-gate")
def pipeline(metric: float = 0.0, minimum: float = 0.8):
    evaluate(metric=metric, minimum=minimum)
```

8. Code Explanation

The component has typed scalar inputs and fails visibly when policy fails. The pipeline composes it without hidden notebook state. Real components use immutable base images, URI-based artifacts, resource requests, and separate deployment authorization.

9. Training / Evaluation

Unit-test component logic locally, then integration-test compiled specs in a namespace. Validate artifact schemas/lineage, retry/idempotency, cache correctness, resource limits, failure propagation, gates, and service-account permissions.

10. Complexity and Cost

Each task adds pod scheduling/image-pull startup, making Kubeflow inefficient for millisecond tasks. Remote artifacts add I/O. Parallelism reduces time but consumes cluster quota. Cache expensive pure steps; coarsen tiny tasks.

11. Common Use Cases

Reusable training pipelines, scheduled retraining, distributed training launch, hyperparameter tuning, batch preprocessing, and governed model promotion.

12. Common Mistakes

Huge data as parameters, mutable images, hidden network dependencies with caching, no resource requests, monolithic notebooks, deployment credentials in training tasks, and treating pipeline success as model approval.

13. Edge Cases / Limitations

Installation/upgrade/RBAC are substantial; local debugging differs from cluster; cache can return stale results; Kubernetes startup dominates small workloads; component APIs vary between Kubeflow versions.

14. Variations

Kubeflow Pipelines focuses workflows; Katib focuses tuning; training operators manage distributed jobs; KServe handles serving. Managed cloud pipeline services reduce platform operations. Learn concepts over product memorization.

15. Related Topics

Kubernetes is substrate; MLflow can track runs/register models; DVC versions file inputs; Ray can run distributed compute inside tasks; CI compiles/submits specs.

16. Interview Questions

1. **Why Kubeflow?** Kubernetes-native repeatable ML workflows and lineage.
2. **Component?** Containerized typed unit of work.
3. **How pass large data?** Object-store URI/artifact.
4. **Cache danger?** Hidden mutable dependencies invalidate keys.
5. **Pipeline versus CI?** ML runtime DAG versus code integration/release workflow.

6. **Failure recovery?** Idempotent tasks, retries, durable outputs.
7. **Why immutable images?** Reproducible cached/executed behavior.
8. **How secure?** Service accounts, RBAC, network policy, secret references.
9. **When not use?** Small/simple/local workflows without cluster need.
10. **KServe relation?** Separate Kubernetes-native serving layer often used with ecosystem.

17. Practice Tasks

Build two components; pass artifact URI; add conditional gate; test cache invalidation; simulate retry-safe output.

18. Project Ideas

| Project | What/stack/data | Resume value | |---|---| | Churn pipeline | KFP/MLflow/object store; Telco |
Workflow platform | | CV tuning | KFP/Katib/PyTorch; CIFAR | Distributed experiments | | RAG refresh |
ingest/embed/evaluate/index; docs | LLM pipeline |

19. Quick Revision

Containerized ML DAGs on Kubernetes. Formula: critical path controls duration. Metrics: pipeline success/time/cache hit/cost plus model gates. Trap: hidden dependencies under caching. One-liner: "Components exchange typed metadata and artifact references, not shared notebook state."

20. Final Cheat Sheet

Input/output: pipeline spec + artifacts to versioned outputs/metadata. Knobs: images, resources, cache, retries, conditions. Pros: scalable repeatable DAGs; cons: Kubernetes overhead/complexity.

Ray

1. Overview

Ray is a distributed Python framework built around remote tasks, stateful actors, and an object store. Its ecosystem supports data processing, training, tuning, reinforcement learning, and serving. It is useful when Python workloads need scale without rewriting into a separate dataflow language.

2. Intuition

Mark functions as jobs and classes as long-lived remote workers; Ray schedules them across available machines and moves object references between them.

3. Prerequisites

Python concurrency, serialization, cluster resources, tasks/actors, fault tolerance, distributed training, object stores, and scheduling.

4. Core Concepts

Concept	Meaning	Interview angle
Task	Stateless remote function	Retry/idempotency
Actor	Stateful remote class	Long-lived model/cache
ObjectRef	Future/reference in object store	Avoid driver materialization
Resources	CPU/GPU/custom labels	Scheduling declaration
Placement group	Co/anti-location bundles	Distributed training topology
Ray Data/Train/Tune/Serve	Higher-level libraries	Use correct abstraction

5. Algorithm / Working Process

Identify genuinely parallel/stateless work or stateful workers; declare remote functions/classes and resource needs; put large shared immutable objects once; submit tasks returning references; compose dependencies without ray.get too early; bound concurrency/backpressure; design retries/idempotency/checkpoints; observe dashboard and scale cluster.

6. Mathematical Foundation

Amdahl's Law:

$$\text{speedup}(N) = 1 / ((1-P) + P/N)$$

where P is parallel fraction. Real time also includes serialization, scheduling, network, stragglers, and object spilling; distributed execution helps only when task work dominates overhead.

7. Practical Implementation

```
import ray

ray.init()

@ray.remote(num_cpus=1)
def score_partition(model_ref, rows):
    model = ray.get(model_ref)
    return model.predict(rows)

model_ref = ray.put(model)
refs = [score_partition.remote(model_ref, part) for part in partitions]
predictions = ray.get(refs)
```

8. Code Explanation

The model is placed once and referenced rather than serialized from the driver for each submission. Partitions execute independently. For repeated model use, actors can load one model per worker. Avoid passing huge

Python objects or calling `ray.get` inside submission loops.

9. Training / Evaluation

First verify single-process correctness. Test retry/idempotency, worker/node loss, object spilling, deterministic partitioning, checkpoint recovery, resource declarations, scaling efficiency, and numerical equivalence.

10. Complexity and Cost

Parallel ideal is T/N ; overhead can dominate small tasks. Object store memory, network transfer, and duplicated actor weights are major costs. Choose coarse tasks, locality, bounded in-flight refs, and autoscaling policies.

11. Common Use Cases

Hyperparameter search, distributed data preprocessing, batch inference, multi-GPU training, RL simulation, and scalable model serving.

12. Common Mistakes

Tiny remote tasks, `ray.get` immediately after each submit, undeclared GPUs, huge closure capture, non-idempotent retries, driver collecting all outputs, unbounded task submission, and assuming distributed means faster.

13. Edge Cases / Limitations

Object-store pressure spills to disk; actor failure loses uncheckpointed state; heterogeneous clusters need constraints; network partitions complicate recovery; some tightly coupled HPC workloads fit MPI better.

14. Variations

Tasks for stateless maps, actors for state, Ray Data for datasets/backpressure, Train for distributed learners, Tune for search, Serve for endpoints. Dask/Spark emphasize dataframes/dataflow; Ray emphasizes general Python tasks/actors.

15. Related Topics

Distributed serving uses Ray Serve; Kubeflow can orchestrate Ray jobs; Kubernetes provides cluster substrate; batch inference maps partitions; distributed training uses collective communication.

16. Interview Questions

1. **Task/actor?** Stateless invocation/stateful long-lived worker.
2. **ObjectRef?** Future-like handle to distributed object.
3. **Why `ray.put`?** Share one large immutable object reference.
4. **Why not immediate `ray.get`?** It serializes execution.
5. **Resource annotations?** Scheduler reservations/placement.
6. **Retry concern?** Side effects must be idempotent.
7. **Backpressure?** Bound in-flight work when producers outrun consumers.
8. **When actor for model?** Expensive load reused across requests/tasks.

9. **Why slower than local?** Serialization/scheduling/network/stragglers.
10. **Ray versus Spark?** General tasks/actors versus optimized dataflow/SQL ecosystem.

17. Practice Tasks

Parallelize partitions; compare task granularity; convert to actors; inject worker failure; cap in-flight references.

18. Project Ideas

| Project | What/stack/data | Resume value | |---|---| | Distributed embedder | Ray Data/HF; text corpus |
 Scalable inference | | Tune service | Ray Tune/MLflow; Adult | HPO | | RL simulator | Ray/RLlib; Gymnasium |
 Research systems |

19. Quick Revision

Distributed Python tasks, actors, object references. Formula: Amdahl's Law. Metrics: speedup, utilization, spill, task overhead, failures. Trap: tiny tasks and eager ray.get. One-liner: "Parallelize coarse independent work and keep data as references."

20. Final Cheat Sheet

Input/output: remote calls to object refs/results. Knobs: resources, task size, actors, concurrency, retries. Pros: flexible Python scale; cons: cluster/debug/data-movement complexity.

Distributed Serving

1. Overview

Distributed serving spreads inference across processes, nodes, accelerators, and sometimes pipeline/tensor/model shards. It supports high traffic or models too large for one device while preserving availability, consistency, and bounded latency.

2. Intuition

A single checkout scales into a coordinated supermarket: route requests, keep popular stations staffed, batch compatible work, and prevent one slow station from blocking all customers.

3. Prerequisites

Model serving, queues, load balancing, batching, sharding, replication, GPU communication, autoscaling, fault tolerance, and observability.

4. Core Concepts

Concept	Meaning	Interview angle
Data parallel serving	Full replica per worker	Throughput scaling

Concept	Meaning	Interview angle
Model/tensor parallel	Model split across devices	Large-model latency/communication
Pipeline parallel	Layers partitioned	Bubbles/microbatches
Router	Chooses replica/model version	Load/locality/stickiness
Dynamic batching	Queue compatible requests briefly	Throughput-tail tradeoff
Backpressure	Bound queues/admit load	Prevent collapse
KV/cache locality	Route session to cached state	LLM efficiency

5. Algorithm / Working Process

Estimate SLO/load/model memory; choose replicas if model fits, sharding if not; pin version/runtime; warm replicas/shards; route by availability/locality and deadline; queue with cap; dynamically batch compatible shapes/deadlines; execute collective/shard operations; stream/postprocess; propagate cancellation; observe every stage; autoscale or shed load; roll version consistently.

6. Mathematical Foundation

```
throughput_cluster <= min(arrival_capacity,
                          replicas * throughput_replica,
                          network_or_shared_dependency_capacity)
model_memory_per_device approximately weights/shards
+ KV_cache + activations + runtime
```

For pipeline stages with times t_i , steady throughput is limited by $\max(t_i)$; latency includes $\sum(t_i)$, communication, and queueing. Tail latency worsens with fan-out because all shards must finish.

7. Practical Implementation

```
from collections import deque

class BoundedBatcher:
    def __init__(self, max_batch=8, max_queue=128):
        self.max_batch, self.queue = max_batch, deque(maxlen=max_queue)

    def submit(self, request):
        if len(self.queue) == self.queue.maxlen:
            raise RuntimeError("overloaded")
        self.queue.append(request)

    def next_batch(self):
        return [self.queue.popleft()
                for _ in range(min(self.max_batch, len(self.queue)))]
```

8. Code Explanation

The bounded queue rejects overload instead of allowing unlimited latency/memory. `next_batch` creates a capped batch. Production batchers wait a small maximum interval, group compatible shapes/sequence lengths, return per-request futures, and propagate deadlines.

9. Training / Evaluation

Validate numerical parity, shard loss/recovery, rolling version consistency, cold/warm starts, skewed lengths, burst load, cancellation, batch fairness, failover, and dependency saturation. Report p50/p95/p99 time-to-first-token and inter-token latency for LLMs.

10. Complexity and Cost

Replication multiplies weight memory; sharding adds network collectives. Batching improves FLOP utilization but adds queue wait. Autoscaling GPUs is slow and expensive, so maintain headroom. Quantization and cache management often provide more gain than extra replicas.

11. Common Use Cases

High-QPS rankers, large language models, multi-model endpoints, vision fleets, recommendation inference graphs, and geographically distributed APIs.

12. Common Mistakes

Unbounded queues, random routing that destroys cache locality, mixed versions in a shard group, CPU-only scaling signals, no admission control, synchronized cold starts, oversized batches hurting p99, and ignoring feature/network bottlenecks.

13. Edge Cases / Limitations

Long prompts create head-of-line blocking; one failed shard can fail a request; hot tenants cause unfairness; multi-region routing adds consistency/data-residency issues; stateful streams complicate failover.

14. Variations

Replica serving, tensor parallel, pipeline parallel, expert parallel for MoE, disaggregated prefill/decode for LLMs, and multi-model serving. Use replicas first when model fits and measured load demands scale.

15. Related Topics

Ray Serve/KServe/Triton/vLLM implement pieces; Kubernetes schedules resources; compression changes memory/compute; monitoring and tracing diagnose stages.

16. Interview Questions

1. **Replica versus shard?** Full model copies versus pieces across devices.
2. **Why dynamic batch?** Increase utilization under variable arrivals.
3. **Batch tradeoff?** Throughput versus queue/tail latency.
4. **Backpressure?** Slow/reject admission to bound system state.

5. **Why cache-aware routing?** Reuse LLM KV/model caches.
6. **Autoscale signal?** Queue, in-flight, latency, accelerator load.
7. **Fan-out tail issue?** Completion waits for slowest dependency.
8. **Model too large?** Quantize/offload/shard, then evaluate cost/latency.
9. **Rolling sharded model?** Keep compatible shard groups and atomic routing versions.
10. **How prevent noisy neighbor?** Tenant quotas, fair queues, isolation.

17. Practice Tasks

Implement bounded batcher; simulate latency/throughput; route sessions sticky; inject shard failure; calculate memory capacity.

18. Project Ideas

| Project | What/stack/data | Resume value | |---|---| | Dynamic batching service | Ray Serve/PyTorch; images |
Serving optimization | | Distributed LLM endpoint | vLLM/K8s; small LLM | GenAI systems | | Multi-tenant
router | quotas/cache locality; synthetic | Platform design |

19. Quick Revision

Scale with replicas when model fits; shard when it does not; bound queues and batch under deadlines. Formula: bottleneck capacity and per-device memory. Trap: scaling model while feature/network bottleneck remains. One-liner: "Distributed serving is a queueing and memory problem before it is a cluster-size problem."

20. Final Cheat Sheet

Input/output: distributed requests to assembled predictions/tokens. Knobs: replicas/shards, batch wait/size, queue, routing, autoscaling. Pros: scale/large models; cons: communication, tail latency, failures, cost.

Model Compression Pipeline

1. Overview

A model compression pipeline converts an accurate source model into a smaller/faster deployable artifact while measuring quality, latency, memory, energy, compatibility, and slice regressions. Common techniques are quantization, pruning, knowledge distillation, low-rank factorization, compact architecture selection, and graph/runtime optimization.

2. Intuition

Compression packs a suitcase for a strict airline limit. Removing weight blindly may discard essentials; a pipeline repeatedly packs, measures, and verifies the suitcase on the actual airline—target hardware and runtime.

3. Prerequisites

Neural networks, tensor shapes, numerical precision, calibration datasets, loss functions, fine-tuning, profiling, ONNX/runtime export, and deployment validation.

4. Core Concepts

Technique	Meaning / example	Why/interview angle
Quantization	FP32 → FP16/INT8/INT4	Scale/zero-point, PTQ versus QAT
Pruning	Remove weights/channels/heads	Unstructured sparsity needs hardware support
Distillation	Student matches teacher	Temperature and blended loss
Low-rank	$W \approx AB$, rank r	Parameter/compute reduction
Export/fusion	ONNX, fused operators	Runtime compatibility
Calibration	Representative activations for ranges	Distribution coverage
Acceptance gate	Pareto quality-latency-memory	Never optimize file size alone

5. Algorithm / Working Process

Freeze source version, evaluation set/slices, and hardware SLO; benchmark source end-to-end; choose simplest technique meeting constraint; create representative calibration/fine-tuning data without test leakage; compress; optionally fine-tune/distill; export target format; verify numerical/operator parity; benchmark warm/cold latency, throughput, peak memory, size, power, and slices on target hardware; register compressed variant linked to source; shadow/canary and monitor.

6. Mathematical Foundation

Affine quantization:

```
q = clip(round(x/s) + z, q_min, q_max)
x_hat = s(q - z)
s = (x_max - x_min)/(q_max - q_min)
```

Distillation with temperature T :

```
L = alpha * CE(y, student_logits)
+ (1-alpha) * T^2 * KL(
    softmax(teacher_logits/T) ||
    softmax(student_logits/T))
```

Low-rank factorization $W_{(m \times n)} \approx A_{(m \times r)} B_{(r \times n)}$ changes parameters from mn to $r(m+n)$. Compression ratio = original bytes/compressed bytes.

7. Practical Implementation

```
import torch

def dynamic_int8(model):
    model = model.eval().cpu()
    return torch.ao.quantization.quantize_dynamic(
        model, {torch.nn.Linear}, dtype=torch.qint8
    )

@torch.inference_mode()
def max_output_error(fp_model, int8_model, sample):
    a = fp_model.eval().cpu()(sample.cpu())
    b = int8_model(sample.cpu())
    return float((a - b).abs().max())

# compressed = dynamic_int8(model)
# assert max_output_error(model, compressed, calibration_batch) < tolerance
```

8. Code Explanation

Dynamic quantization converts supported Linear weights to INT8 and calculates activation quantization at runtime, making it a strong CPU baseline with little code. `inference_mode` removes autograd overhead. Output tolerance is only a smoke check; full task/slice evaluation and target latency are mandatory.

9. Training / Evaluation

Use representative calibration data from the training/validation population, never tune on the final test. Evaluate top-line and rare/safety slices, calibration, ranking, robustness, and output parity. Benchmark end-to-end with warm-up, realistic batch/sequence lengths, many repetitions, p50/p99, peak RSS/VRAM, artifact size, throughput, power, and cold start. QAT or distillation needs validation-based early stopping.

10. Complexity and Cost

INT8 weights are theoretically 4× smaller than FP32, but metadata/unquantized layers reduce end-to-end gain. Low-rank compute changes $O(mn)$ to $O(r(m+n))$. Unstructured pruning may not speed dense hardware. Distillation costs teacher inference plus student training; QAT costs fine-tuning. Always measure the deployed runtime.

11. Common Use Cases

Mobile/edge vision, CPU NLP, cheaper LLM inference, browser/on-device models, high-QPS rankers, and reduced cold start/network transfer.

12. Common Mistakes

Reporting file size instead of latency, calibrating on unrepresentative data, test-set tuning, assuming sparsity speeds hardware, export without parity, average-only benchmarking, comparing different batch sizes, and losing tokenizer/preprocessing.

13. Edge Cases / Limitations

Outliers damage low-bit ranges; sensitive layers may need mixed precision; unsupported operators fall back and erase gains; small models can become slower from quantize/dequantize overhead; generative error may accumulate; target hardware may lack kernels.

14. Variations

PTQ is fastest and first choice; QAT recovers difficult INT8 accuracy; weight-only 8/4-bit helps LLM memory; structured pruning gives deployable dense speedups; distillation trains a smaller architecture; LoRA reduces adaptation storage but does not automatically compress the base runtime. All are interview-relevant; kernel-aware research is advanced.

15. Related Topics

Compression creates a new model version, registry records source linkage, Docker/runtime packages kernels, distributed serving uses lower memory, shadow validates parity, and monitoring catches slice regressions.

16. Interview Questions

1. **PTQ versus QAT?** Quantize after training versus simulate quantization during fine-tuning.
2. **Scale/zero point?** Map real range to integer grid.
3. **Dynamic/static quantization?** Activations quantified at runtime versus calibrated fixed ranges.
4. **Why T^2 in distillation?** Preserve gradient scale when temperature softens distributions.
5. **Structured/unstructured pruning?** Remove channels/blocks versus individual weights.
6. **Why no sparse speedup?** Runtime/hardware may execute dense kernels.
7. **Calibration set?** Representative unlabeled/labeled examples for activation ranges, separate from final test tuning.
8. **What benchmark?** Task/slices, p50/p99, throughput, memory, size, energy, cold start on target.
9. **Quantized slower why?** Small workload, unsupported fallback, conversion overhead, poor kernels.
10. **How release?** New immutable linked artifact, full gates, shadow/canary, monitor.

17. Practice Tasks

Quantize a linear model; measure size and p99; create outlier calibration failure; distill teacher/student; export ONNX and compare outputs.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Edge vision optimizer	FP32/INT8/ONNX benchmarking	PyTorch, ONNX; CIFAR	Hardware-aware CV
NLP distillation	Teacher to compact classifier	HF; AG News	DL compression
LLM compression lab	8/4-bit quality-memory-cost	HF/quant runtime; QA set	GenAI efficiency

19. Quick Revision

Key idea: optimize a measured Pareto frontier on target hardware. Formulas: affine quantization, distillation loss, low-rank parameter count. Metrics: task/slices, p99, throughput, memory, bytes, energy/cost. Trap: theoretical compression without real speedup. One-liner: "A compressed model is a new release candidate, not a file conversion."

20. Final Cheat Sheet

Item	Summary
Definition	Governed conversion to smaller/faster deployable model
Input/output	Source artifact + representative data → validated compressed artifact
Steps	Baseline, choose, calibrate/train, export, parity, benchmark, register, roll out
Hyperparameters	bits, per-channel/group size, clipping, sparsity, rank, alpha/T
Metrics	Quality/slices, calibration, p99, QPS, RAM/VRAM, size, energy
Pros/cons	Lower cost/latency / accuracy, kernel, and workflow risk
Best use	Edge, high-QPS, large-model, memory/cost constrained serving